Đề cương nghiên cứu GRACE: tái lập và chẩn đoán trên LLM open-weight

28/10/2026

Tóm tắt điều hành

Khuyến nghị: dừng theo đuổi H1, H2, H3 như các "cải tiến GRACE" và chuyển toàn bộ kết quả đã có thành một **nghiên cứu tái lập có chẩn đoán** (diagnostic replication) của GRACE trên LLM open-weight. Đây là hướng rẻ nhất vì khoảng 80% dữ liệu cần thiết đã nằm trong kho artifact của nhóm, và có giá trị khoa học vì nó trả lời một câu hỏi cộng đồng đang đặt ra: công thức *graph + demonstration truy hồi* có hiệu quả ngoài GPT-4 hay không, và nếu không thì vì sao.

Ba phát hiện đã có đủ sức nặng để làm xương sống của một bài báo:

1. **GRACE không chuyển sang LLM open-weight.** Với Qwen3-Coder-480B và DeepSeek-V4.1-Flash, full GRACE đạt MCC 0,02–0,10 trên toàn bộ test Devign (2.732 mẫu) và ReVeal (2.274 mẫu), tức mức ngẫu nhiên — trùng với kết luận của tái lập độc lập RAG4SVD (Kaniewski et al., 2026) trên một khung benchmark chung; bộ phân loại hằng "luôn đoán vulnerable" có F1 62,95% trên Devign, cao hơn mọi F1 mà GRACE đạt được.
2. **Một hồi quy logistic trên embedding CodeT5 đóng băng thắng GRACE với cách biệt lớn.** Gate của H3-B đạt MCC 0,30 (Devign) và 0,37 (ReVeal); chênh lệch so với full GRACE là −0,23 và −0,27 với khoảng tin cậy 95% không chứa 0. Mọi cách phối hợp LLM với bộ phân loại (router, selector, stacking) đều không vượt gate.
3. **Demonstration hoạt động như gợi ý nhãn, không phải như ví dụ để học.** Đảo nhãn demo làm 31% dự đoán đổi theo (biến động tự nhiên 4%); cặp tương phản giảm mức này còn 13–18% nhưng không cải thiện MCC; chấm mù cho thấy 60% target Devign không có demo nào liên quan về cơ chế.

Việc còn thiếu để công bố, theo thứ tự ưu tiên:

- Một lượt chạy cuối **thống nhất trên một LLM tự host có logprob** (Qwen3-Coder-30B-A3B-Instruct qua vLLM hoặc llama.cpp; dự phòng Qwen2.5-Coder-32B), để mọi đối chứng cùng điều kiện và có điểm số liên tục thay vì chỉ nhãn 0/1.
- Bốn đối chứng rẻ còn thiếu trên full test: không graph (B0), graph xáo trộn, demo ngẫu nhiên, demo bỏ nhãn.
- Đánh giá không ngưỡng (AUROC, AUPRC) để tách "khả năng phân biệt" khỏi "điểm hoạt động", giải quyết dứt điểm tranh luận recall tăng – FP tăng của cặp tương phản.
- Phân tích **giá trị bổ sung** của LLM so với mốc có giám sát, tính offline từ dự đoán đã có: cặp meta-model ghép đôi có/không đặc trưng LLM (P_pre/P_post), bảng sửa–hỏng theo lớp đặt cạnh expert hằng và expert ngẫu nhiên, đường chất lượng theo ngân sách gọi LLM.
- Thống kê ghép cặp với **biên tác dụng δ khai báo trước** (để phân biệt "không có lợi ích đáng kể" với "chưa đủ bằng chứng"), ba seed, và pre-registration trước lượt chạy cuối.
- Viết bài theo khung RENE/ESEM: tái lập + chẩn đoán + mốc có giám sát.

Thời gian và chi phí: khoảng 8 tuần đến bản thảo hội thảo (track RENE của ICSME/SANER/ICPC, hoặc ESEM); thêm 3–4 tháng để mở rộng thành bài tạp chí Q1 (Empirical Software Engineering hoặc Journal of Systems and Software) với Big-Vul/PrimeVul, nhiều cỡ model, demo cặp vá lỗi thật và kiểm toán nhãn. Chi phí tính toán: một GPU thuê (A100 hoặc L40S) khoảng 20–40 giờ, tương đương

30–60 USD, hoặc 0 USD nếu chạy bản GGUF trên Kaggle 2×T4 như nhóm đã thử; không phụ thuộc API trả phí.

Các đề xuất H1, H2, H3-A, H3-B và cặp demo tương phản không bị bỏ đi: chúng trở thành **đối chứng và mốc so sánh** trong khung chẩn đoán (Mục 2), nên công sức đã bỏ ra đều được dùng trong bài.

Đề cương này đã được đối chiếu với một phương án suy luận khác cho cùng bài toán: lấy H3-B làm nền và biến bài báo thành một nghiên cứu về *giá trị gia tăng của nhánh LLM so với bộ phân loại nhẹ* (learning to defer), xác nhận trên một cohort PrimeVul khóa trước. Nhiều yếu tố của phương án đó đã được tiếp thu (cặp meta-model ghép đôi, phân rã sửa–hỏng theo lớp, routing theo ngân sách, biên tác dụng δ, kiểm tra toàn vẹn prompt, quy tắc dừng); bảy điểm không được chấp nhận cùng lý do được trình bày ở Mục 12.

1. Bối cảnh và động cơ nghiên cứu

Phát hiện lỗ hổng ở mức hàm (function-level vulnerability detection) là bài toán phân loại nhị phân: cho một hàm C/C++, trả lời hàm đó có lỗ hổng hay không. Trong năm năm qua, hướng chủ đạo là học sâu có giám sát trên các bộ dữ liệu Devign, ReVeal, Big-Vul: mô hình đồ thị (Devign, ReVeal, IVDetect) và mô hình ngôn ngữ mã nguồn tinh chỉnh (CodeBERT, LineVul, UniXcoder). Kết quả của hướng này ổn định nhưng khiêm tốn: trên Devign, các mô hình tốt nhất đạt accuracy 62–66% và F1 60–66%, tương đương MCC khoảng 0,25–0,35.

Từ 2023, các nhóm nghiên cứu thử thay huấn luyện bằng prompting LLM. GRACE (Lu et al., *Journal of Systems and Software*, 2024) là một trong những công trình được trích dẫn nhiều nhất theo hướng này. Công thức của GRACE gồm ba thành phần: (1) đưa thông tin đồ thị thuộc tính mã (CPG từ Joern) vào prompt; (2) truy hồi một demonstration tương đồng từ tập train (CodeT5 + Jaccard + AST) kèm nhãn; (3) prompt có vai trò và miền. Với GPT-4, bài báo công bố F1 65,11% trên Devign và 43,13% trên ReVeal, và kết luận rằng graph đem lại cải thiện tương đối 15–74% F1, còn demonstration truy hồi đem lại 32% so với zero-shot.

Công thức này đã được nhiều nhóm kế thừa, ví dụ bài *selective GRACE* của Gia Khanh Tran và Minh Dung Vo (APSIPA ASC 2026) xây một bộ lọc trước để tiết kiệm lượt gọi GRACE. Tất cả đều giả định rằng kết quả của GRACE là thật và chuyển được sang các LLM khác. Mãi đến tháng 9/2026 mới có một tái lập độc lập trong khung benchmark chung (RAG4SVD; Kaniewski et al., 2026) cho thấy hiệu năng công bố của GRACE và năm hệ RAG khác không chuyển sang LLM open-weight; nhưng tái lập đó dừng ở mức đầu-cuối và theo tầng pipeline (trừu tượng hóa đầu vào, truy hồi, phát hiện), không trả lời vì sao: LLM có đọc graph không, demonstration tác động bằng nội dung hay bằng nhãn, và LLM đứng ở đâu so với một bộ phân loại rẻ.

Có ba lý do để nghi ngờ giả định đó:

- **Bằng chứng ngoài nhóm.** Các nghiên cứu thực nghiệm quy mô lớn (Ullah et al., IEEE S&P 2024; Steenhoek et al., 2024; Ding et al., ICSE 2025) cho thấy LLM, kể cả GPT-4, chỉ đạt mức gần ngẫu nhiên khi phát hiện lỗ hổng thực tế, và các bộ dữ liệu Devign/Big-Vul có tỷ lệ nhãn sai đáng kể (Croft et al., ICSE 2023); một bộ phân loại chỉ dùng code metrics cổ điển đạt ngang các LLM tốt nhất, và dự đoán của LLM biến thiên theo chính các metric đó (Weissberg et al., ICSE 2026). Chính bài GRACE cũng nêu trong *Threats to validity* khả năng GPT-4 đã thấy dữ liệu test khi tiền huấn luyện.
- **Bằng chứng trong nhóm.** Sau hai tháng, nhóm đã tái triển khai GRACE theo hai bản độc lập, chạy trên ba LLM khác nhau (Gemma-4-26B, Qwen3-Coder-480B, DeepSeek-V4.1-Flash), và

thử năm hướng cải tiến. Không cấu hình nào vượt MCC 0,10 trên full test. Trong khi đó, một bộ phân loại tuyến tính trên embedding CodeT5 đạt MCC 0,30–0,37.

- **Khoảng cách với cách đánh giá chuẩn.** Bài GRACE báo cáo Accuracy/Precision/Recall/F1, không có MCC, không có đối chứng hằng, không có khoảng tin cậy. Trên Devign, bộ phân loại "luôn đoán vulnerable" đã có F1 ≈ 62%, nên F1 65,11% của bài báo chỉ hơn mốc sàn khoảng 3 điểm.

Động cơ của nghiên cứu vì thế không phải là "làm GRACE tốt hơn một chút", mà là trả lời rõ ràng, có bằng chứng, một câu hỏi mà người làm sau cần biết trước khi xây tiếp trên GRACE: *với LLM open-weight và cách đánh giá chuẩn, graph và demonstration truy hồi có thực sự giúp LLM phát hiện lỗ hổng không, và nếu không thì cơ chế nào giải thích điều đó?* Một câu trả lời âm tính được chứng minh chặt chẽ, kèm cơ chế giải thích và mốc so sánh mạnh, là đóng góp được các track *Reproducibility Studies and Negative Results* (RENE) và các tạp chí thực nghiệm như EMSE đòi hỏi.

2. Hiện trạng thực nghiệm của nhóm và đánh giá các đề xuất

Kết luận của mục này: nhóm đã có gần đủ bằng chứng cho một bài tái lập có chẩn đoán, nhưng không đề xuất nào trong H1–H3 đủ bằng chứng để gọi là cải tiến. Từng đề xuất được giữ lại dưới vai trò mới.

2.1. Kho bằng chứng đã có (tính đến 24/10/2026)

Bằng chứng	Số liệu chính	Dùng cho
Full GRACE tái triển khai, Qwen3-Coder-480B (Vertex), full test	Devign: baseline MCC 0,069, Full V1+V2 0,072; ReVeal: 0,023 và 0,051	RQ1 (backbone thứ hai)
Full GRACE, DeepSeek-V4.1-Flash, full test	Devign MCC 0,067 (full_grace), 0,054 (H2-guarded); ReVeal 0,097 và 0,066	RQ1, RQ3 (backbone thứ ba)
Bộ phân loại có giám sát trên embedding CodeT5 đóng băng (gate của H3-B)	MCC 0,296 (Devign, τ = 0,53) và 0,371 (ReVeal, τ = 0,14); ΔMCC full GRACE − gate = −0,229 [−0,277; −0,182] và −0,273 [−0,350; −0,196]	RQ5 (mốc có giám sát)
Phối hợp LLM + bộ phân loại: router, post-selector, stacking	Không arm nào vượt gate_tuned; mọi CI ΔMCC chứa 0 hoặc âm	RQ5
H1-V2 Security Projection (metric learning cho retrieval)	Proxy retrieval tăng nhẹ; downstream ReVeal ΔMCC +0,002, CI chứa 0	RQ2 (truy hồi tốt hơn không đổi kết quả)
H3-A Calibrated Cascade	Bỏ qua 6/2.732 mẫu Devign (0,22%), 0 mẫu ReVeal	Thảo luận, một câu
B2 vs B3 (cặp tương phản) vs B3-NoDiff trên dev 100 mẫu, Qwen3-Coder-480B	Devign: MCC −0,03 → −0,11, tỷ lệ đoán 1 tăng 29% → 66%; ReVeal: 0,145 → 0,189; CI ΔMCC chứa 0	RQ2, RQ4 (cần chạy lại full test)
Độ nhạy nhãn demo trên dev	Net label sensitivity 31% (B2), biến động tự nhiên	RQ2 (phát

Bằng chứng	Số liệu chính	Dùng cho
100 mẫu	4%; B3: 18% Devign, 13% ReVeal	hiện chính)
Chấm mù 50 + 50 target, hai người chấm độc lập	κ = 0,32 và 0,33; Devign: 30/50 “Neither”; ReVeal: security-aware 21 vs baseline 10	RQ6
Kiểm toán trùng lặp	Trùng chính xác: Devign 108 hàm (3,95%), ReVeal 52 (2,29%); ID overlap 0	Vệ sinh dữ liệu
Parser đã sửa, phân rã invalid	9.307 invalid trong raw cũ (9.206 phản hồi rỗng từ provider); reparse 0 nhãn đổi	Giao thức
Đối chứng hằng	Devign: F1 luôn-vulnerable 62,95%; ReVeal 18,3%	Mọi bảng

2.2. Đánh giá từng đề xuất và vai trò mới

Đề xuất	Ý tưởng	Kết quả tới 24/10	Nhận định	Vai trò trong đề cương
H1-V1, H1-V2	Học biểu diễn để chọn demo sát nhãn hơn	Proxy P@5 tăng 0,009; downstream không đổi	Đúng hướng nếu demo tác động qua nội dung; nhưng độ nhạy nhãn 31% cho thấy demo tác động qua nhãn, nên chọn demo tốt hơn không đổi kết quả	Một bảng trong RQ2; không phát triển tiếp
H2 Guarded Evidence	Đóng gói graph thành cụm thao tác–phụ thuộc–guard theo ngân sách	Chạy full test DeepSeek, MCC 0,054 và 0,066; chưa có prefix control cùng điều kiện	Kỹ thuật tốt, nhưng không thể có tác dụng nếu LLM không đọc graph; phải kiểm tra điều đó trước (graph xáo trộn)	Một arm trong RQ3 (B1-H2), chạy cùng B1-prefix và B1-shuffle; pipeline đã có nên chi phí thấp
H3-A Calibrated Cascade	Gate có hiệu chuẩn bỏ qua mẫu chắc an toàn	Tiết kiệm 0,22% query	Ràng buộc 1% làm vùng tự quyết rỗng; không có giá trị thực nghiệm	Một câu trong thảo luận
H3-B Benchmark Ensemble	Nhiều view + meta-model phối hợp với GRACE	Winner là gate-only; LLM không bổ sung tín hiệu	Sản phẩm giá trị nhất của nhóm, nhưng đúng tên là “mốc có giám sát” chứ không phải cải tiến GRACE	Trục RQ5: gate là mốc mạnh; 34 đặc trưng nền, 4 đặc trưng expert và benefit-router của H3-B được dùng lại cho cặp P_pre/P_post, bảng sửa–hỏng và đường routing theo ngân sách
Security-	Cặp demo 1	Recall tăng, FP	Là đúng một nửa: cân bằng nhãn	Trục RQ2 và RQ4;

Đề xuất	Ý tưởng	Kết quả tới 24/10	Nhận định	Vai trò trong đề cương
aware retrieval + contrastive ICL	vulnerable + 1 safe có điều kiện kiểm tra khác nhau	tăng, MCC không đổi; độ nhạy nhãn giảm	làm giảm sao chép nhãn, nhưng chưa biết khả năng phân biệt có tăng hay chỉ điểm hoạt động dịch chuyển; đánh giá không ngưỡng sẽ trả lời	chạy lại trên full test với logprob
Chấm mù retrieval	Rubric sink–biến–guard–dòng mã	100 target, κ ≈ 0,32	Đây là dữ liệu ít bài nào có; cần nối với kết quả dự đoán	Trục RQ6; mở rộng lên 100 + 100 cho tạp chí
Chuyển backbone sang Gemini 1.5 hoặc vLLM riêng	Thay endpoint Qwen đã ngừng	Chưa chạy	Chọn vLLM tự host: tái lập được, có logprob, không phụ thuộc quota; Gemini chỉ dùng cho kiểm tra model mạnh trên subset ở bản tạp chí	Hạ tầng của lượt chạy cuối
Hoàn thiện H1/H2/H3 thành cải tiến	Thêm đối chứng, benchmark, phân tích chi phí	—	Mỗi hướng cần thêm 3–6 tuần và xác suất ra kết quả dương thấp, vì mọi dấu hiệu hiện có đều chỉ về phía LLM không dùng được thông tin đưa vào	Không chọn; thay bằng khung chẩn đoán

2.3. Bảy điểm trong hồ sơ phải sửa trước khi viết

1. **Đơn vị context chưa thống nhất.** Một bản tái triển khai cắt theo 8.192 byte UTF-8, bản kia theo 8.192 token và giới hạn 25 nút/25 cạnh. Lượt chạy cuối dùng một đơn vị duy nhất là token của tokenizer model, và ghi số token từng phần của mỗi prompt.
2. **Test set đã được xem nhiều lần.** Mọi quyết định chọn ngưỡng, chọn arm của lượt cuối phải làm trên official validation; giả thuyết và cấu hình được đăng ký trước (pre-registration) rồi mới chạy test; bài báo nêu rõ mức độ đã tiếp xúc test. Đăng ký trước chỉ áp dụng cho lượt xác nhận mới; không gọi kết quả lịch sử là "đã đăng ký trước".
3. **API không tất định và không đồng nhất giữa development và test.** Cùng prompt hash, hai lượt gọi Qwen trên Vertex cho nhãn khác nhau ở 6 target; 9.206 phản hồi rỗng từ provider; trong H3-B, expert của development và của test đến từ hai provider/alias khác nhau. Tự host với seed cố định loại bỏ các vấn đề này và cho phép lấy logprob; các kết quả API lịch sử chỉ còn vai trò bằng chứng phụ cho RQ1.
4. **Dev set 100 mẫu không đại diện.** ReVeal dev 50/50 trong khi test có 10% vulnerable; với 100 mẫu, sai số MCC cỡ ±0,2. Các số dev chỉ dùng để chọn cấu hình; mọi kết luận trong bài lấy từ full test.
5. **Các phép so trong H3-B chưa cùng một ngưỡng.** Router/selector dùng gate với ngưỡng chọn trên train, còn mốc gate_tuned dùng ngưỡng chọn trên validation; stacker có 38 đặc trưng và một tầng học thêm mà gate không có. Mọi so sánh ở lượt cuối dùng một bộ phân loại, một ngưỡng và một bộ đặc trưng nền chung; winner đã khóa không được chọn lại bằng test (gate_0_5 có MCC cao hơn gate_tuned trên Devign nhưng winner vẫn giữ).
6. **Số liệu trùng lặp chưa thống nhất.** Hai cách đếm cho hai kết quả khác nhau (108/52 hàm so với 5/63 hàm bị loại khỏi test). Bước 2 làm một kiểm toán duy nhất với một quy tắc chuẩn hóa, báo cả số trùng chính xác lẫn gần trùng, và ghi hash manifest.

7. **Prompt thực gửi chưa được kiểm tra có hệ thống.** Trong hồ sơ cũ có demo bị cắt rỗng và prompt giống nhau nhưng dự đoán khác nhau. Lượt cuối kiểm tra toàn vẹn mọi prompt trước khi gọi LLM (Bước 3): phần giữ lại của code, demo, graph; demo rỗng; rò rỉ nhãn.

3. Vấn đề đặt ra và khoảng trống nghiên cứu

Vấn đề trung tâm: **kết quả của GRACE chưa bao giờ được tách thành phần do phương pháp và phần do GPT-4, và cách đánh giá trong bài không đủ để làm việc đó.** Bài báo cho thấy "GRACE với GPT-4 đạt F1 cao hơn baseline", nhưng không cho biết điều gì xảy ra khi đổi model, không cho biết LLM có thực sự đọc graph hay không, và không cho biết demonstration giúp LLM bằng nội dung hay bằng nhãn đi kèm.

Năm khoảng trống cụ thể mà nghiên cứu này lấp:

#	Khoảng trống	Vì sao quan trọng	Nhóm đã có gì
G1	Chưa có tái lập nào của GRACE trên LLM open-weight đi kèm chẩn đoán cơ chế; tái lập duy nhất (RAG4SVD, 9/2026) chỉ đo đầu-cuối và theo tầng pipeline trên một benchmark chung	Mọi công trình kế thừa đang xây trên một kết quả chưa được giải thích; GPT-4 có thể đã nhìn thấy Devign/ReVeal khi tiền huấn luyện; RAG4SVD xác nhận "không chuyển được" nhưng không nói vì sao	Full test trên ba backbone, MCC ≤ 0,10
G2	Bài gốc không có đối chứng hằng, MCC, khoảng tin cậy; F1 trên Devign chỉ hơn mốc sàn 3 điểm	Đây là lỗi phổ biến trong cả nhánh nghiên cứu, không riêng GRACE; một bài chỉ ra điều này bằng số liệu có giá trị phương pháp luận	Script đánh giá chung, đối chứng hằng, paired CI
G3	Không ai kiểm tra demonstration tác động qua nội dung hay qua nhãn	NLP đã biết LLM sao chép nhãn demo (Min et al., 2022; Wei et al., 2023) nhưng chưa ai đo điều này trong phát hiện lỗ hổng; nếu demo chỉ là gợi ý nhãn thì cả hướng "retrieval tốt hơn" vô nghĩa	Độ nhạy nhãn 31% trên dev; H1 không đổi kết quả
G4	Không ai kiểm tra LLM có đọc graph không dưới ngân sách context thực tế	Bài gốc cắt graph theo 2.000 ký tự, bản tái triển khai theo 25 nút; với CPG vài trăm nút, phần đưa vào gần như không mang thông tin luồng dữ liệu; RQ2 của bài gốc (bỏ graph) không phân biệt được "đọc graph" với "đổi độ dài prompt"	Pipeline H2 và graph prefix đã có; chưa có graph xáo trộn
G5	Chưa có mốc có giám sát rẻ trên cùng benchmark và cùng danh sách ID để định vị GRACE, và chưa ai đo phần tín hiệu LLM bổ sung cho mốc đó bằng đối	Nếu một hồi quy logistic trên embedding đóng băng, huấn luyện vài phút trên CPU, thắng pipeline LLM với cách biệt 0,2–0,3 MCC thì câu hỏi "nên dùng LLM theo cách này không" đã có câu trả lời; Weissberg et al. (ICSE 2026) cho thấy bộ phân loại chỉ dùng code metrics đã ngang LLM,	Gate H3-B và các arm phối hợp

#	Khoảng trống	Vì sao quan trọng	Nhóm đã có gì
	chứng ghép đôi	cần kiểm tra điều đó với GRACE	

Ngoài năm khoảng trống trên, có một vấn đề đo lường riêng của cách dùng LLM ra nhãn 0/1: một prompt mới có thể làm recall tăng và FP tăng cùng lúc (như đã thấy với cặp demo tương phản), và khi đó không thể biết khả năng phân biệt có tăng hay chỉ điểm hoạt động dịch chuyển. Cách giải quyết chuẩn là lấy điểm số liên tục (xác suất token "1" so với "0") và đánh giá không ngưỡng bằng AUROC/AUPRC; điều này đòi hỏi logprob, tức phải tự host model. Đây là lý do kỹ thuật chính khiến lượt chạy cuối phải chuyển sang vLLM.

4. Câu hỏi nghiên cứu, mục tiêu và đóng góp dự kiến

Sáu câu hỏi nghiên cứu, mỗi câu gắn với một khoảng trống ở Mục 3 và một nhóm thí nghiệm ở Mục 7:

RQ	Câu hỏi	Khoảng trống	Thí nghiệm chính
RQ1 – Tái lập	Với LLM open-weight và giao thức chuẩn, GRACE có vượt zero-shot và đối chứng hằng không, và có đạt mức bài gốc không?	G1, G2	B0 → B1 → B2 trên full test, ba backbone
RQ2 – Cơ chế demonstration	Demonstration tác động qua nội dung mã hay qua nhãn đi kèm?	G3	Demo ngẫu nhiên, đảo nhãn, bỏ nhãn, chỉ gợi ý nhãn, cặp tương phản
RQ3 – Graph	LLM có dùng thông tin graph không dưới các ngân sách context thực tế?	G4	Không graph, prefix 25, graph đầy ngân sách, H2 guarded, graph xáo trộn
RQ4 – Phân biệt hay dịch chuyển ngưỡng	Các biến thể prompt khác nhau về khả năng phân biệt (AUROC/AUPRC) hay chỉ về điểm hoạt động?	G2	Điểm logprob cho mọi arm; ngưỡng chọn trên validation
RQ5 – Mốc có giám sát	Một bộ phân loại tuyến tính trên embedding CodeT5 đóng băng (và một bộ chỉ dùng code metrics) so với pipeline LLM như thế nào; LLM có bổ sung tín hiệu cho mốc đó không — sau truy vấn (P_post so với P_pre) và trước truy vấn (routing theo ngân sách)?	G5	SUP-LR, SUP-MLP, SUP-METRIC; cặp P_pre/P_post; sửa–hỏng theo lớp với đối chứng expert hằng/ngẫu nhiên; đường chất lượng theo ngân sách 5/10/25%
RQ6 – Truy hồi và lỗi	Demo truy hồi có liên quan về cơ chế không; liên quan có kéo theo dự đoán đúng không; bao nhiêu "lỗi" thực ra là nhãn nhiễu?	G3, G5	Chấm mù 200 target, phân tích có điều kiện, kiểm toán 100 ca sai

4.1. Mục tiêu

1. Xác định bằng số liệu có khoảng tin cậy liệu công thức GRACE có chuyển sang LLM open-weight hay không.
2. Giải thích cơ chế: tách tác dụng của nhãn demo, nội dung demo và graph bằng các đối chứng thay một thành phần mỗi lần.
3. Định vị giá trị của LLM so với một mốc có giám sát rẻ và so với đối chứng hằng.
4. Để lại một giao thức đánh giá và gói tái lập mà người khác dùng lại được cho mọi phương pháp phát hiện lỗ hổng dựa trên ICL.

4.2. Đóng góp dự kiến

- **C1.** Tái lập độc lập của GRACE trên ba LLM open-weight theo đúng benchmark và giao thức của bài gốc (bổ sung cho tái lập trong khung chung của RAG4SVD), với full test, MCC, đối chứng hằng và khoảng tin cậy ghép cặp.
- **C2.** Một khung chẩn đoán cho phát hiện lỗ hổng bằng ICL: đảo nhãn demo, bỏ nhãn demo, graph xáo trộn, đánh giá không ngưỡng từ logprob. Các phép thử này rẻ và áp dụng được cho bất kỳ phương pháp prompting nào.
- **C3.** Bằng chứng rằng demonstration hoạt động chủ yếu như gợi ý nhãn, và rằng cân bằng nhãn (cặp tương phản) làm giảm sao chép nhãn nhưng không tăng khả năng phân biệt.
- **C4.** So sánh với mốc có giám sát: hồi quy logistic trên embedding đóng băng vượt pipeline LLM; phối hợp không thêm giá trị — đo bằng cặp meta-model ghép đôi (P_pre/P_post), bảng sửa–hỏng theo lớp có đối chứng expert hằng/ngẫu nhiên và đường chất lượng theo ngân sách gọi LLM. Điều này định giá lại các hướng "selective GRACE".
- **C5.** Dữ liệu chấm mù về độ liên quan cơ chế của demo truy hồi, nối với kết quả dự đoán, và kiểm toán nhãn của các ca sai.
- **C6.** Gói tái lập: mã, prompt, dự đoán từng mẫu, logprob, manifest, và bản đăng ký trước.

4.3. Giả thuyết đăng ký trước (pre-registered)

Các dự đoán dưới đây được viết trước lượt chạy cuối và đăng lên OSF (hoặc một commit có dấu thời gian) trước khi chạy test. Kết quả trái dự đoán cũng được báo cáo nguyên vẹn; một giả thuyết bị bác là phát hiện dương tính của bài.

Mã	Dự đoán	Tiêu chí quyết định (biên δ khai báo trước)
H1.1	GRACE (B2) không vượt zero-shot (B0) về khả năng phân biệt trên cả hai dataset	Cận trên CI 95% của ΔMCC(B2 − B0) < 0,03 và của ΔAUROC < 0,02
H1.2	GRACE không đạt mức bài gốc	MCC(B2) < 0,15 trên cả hai dataset; F1(B2) trên Devign thấp hơn F1 luôn-vulnerable
H2.1	Demo tác động qua nhãn	Net label sensitivity (tỷ lệ đổi khi đảo nhãn trừ biến động tự nhiên) ≥ 15 điểm
H2.2	Nội dung demo không mang tín hiệu	Cận trên CI của ΔAUROC(B2 − B2-random) < 0,02 và của ΔAUROC(B2-nolabel − B1) < 0,02; B2-labelonly tái tạo ≥ 80% chênh lệch B2 − B1
H2.3	Cặp tương phản chỉ dịch chuyển điểm hoạt động	Tỷ lệ đoán 1 của B3 cao hơn B2 ≥ 10 điểm; cận trên CI của ΔAUROC(B3 − B2) < 0,02

Mã	Dự đoán	Tiêu chí quyết định (biên δ khai báo trước)
H3.1	LLM không đọc graph	Cận trên CI của ΔAUROC(B1 − B0) < 0,02 và của ΔAUROC(B1 − B1-shuffle) < 0,02; B1-H2 không khác B1 theo cùng tiêu chí
H4.1	Pipeline LLM gần ngẫu nhiên khi đo không ngưỡng	AUROC của mọi arm LLM ≤ 0,62
H5.1	Mốc có giám sát thắng rõ	ΔMCC(SUP-LR − B2) ≥ 0,15 với CI không chứa 0; AUROC(SUP-LR) ≥ 0,68
H5.2	LLM không bổ sung tín hiệu sau truy vấn	Cận trên CI của ΔMCC(P_post − P_pre) < 0,02 (cùng 34 đặc trưng nền, cùng fold, cùng ngưỡng)
H5.3	Không biết trước khi nào nên gọi LLM	Tại ngân sách 10%, Δbalanced accuracy(benefit-router − random) có cận trên CI < 0,01; số ca LLM sửa đúng không vượt khoảng 95% của expert ngẫu nhiên cùng tỷ lệ đoán 1
H6.1	Truy hồi hiếm khi tìm được demo đúng cơ chế; liên quan không kéo theo đúng	Tỷ lệ "Neither" ≥ 40% trên Devign; chênh lệch accuracy giữa nhóm demo liên quan và không liên quan có CI chứa 0

Cách đọc kết quả của mỗi cặp so sánh được cố định trước, để không lẫn "chưa đủ bằng chứng" với "không có tác dụng":

Kết quả của Δ = metric(B) − metric(A)	Kết luận
CI nằm hoàn toàn trên 0 và Δ ≥ δ	Có bằng chứng lợi ích trong giao thức đã kiểm
Cận trên CI < δ	Có bằng chứng loại trừ lợi ích ≥ δ; giả thuyết "không tác dụng" được ủng hộ
CI nằm dưới 0	Có bằng chứng bất lợi
CI chứa 0 và còn chứa mức ≥ δ	Chưa đủ bằng chứng; không kết luận tương đương

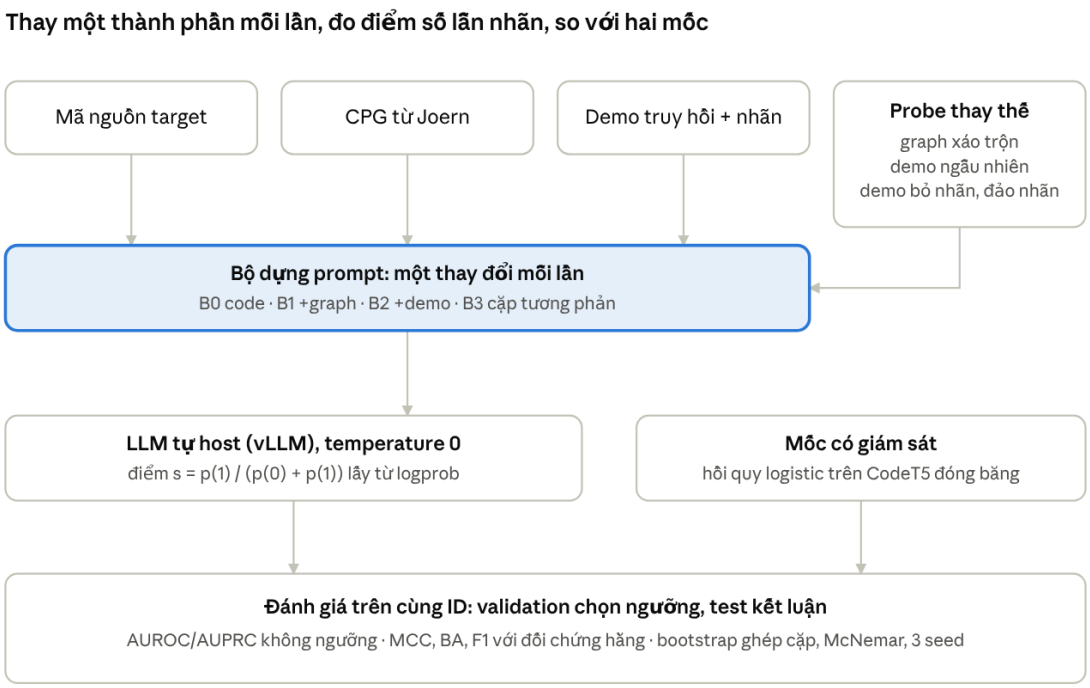
Biên δ (0,03 MCC; 0,02 AUROC; 0,01 balanced accuracy; 0,02 MCC cho cặp meta-model) là mức lợi ích nhỏ nhất mà nhóm cho là đáng trả chi phí một lượt gọi LLM; δ được chốt trong `preregistration.md` và không điều chỉnh sau khi thấy CI.

Nếu H3.1 bị bác (graph xáo trộn làm kết quả xấu đi rõ rệt), đó là bằng chứng LLM có đọc graph và hướng H2 trở nên đáng đầu tư; nếu H2.2 bị bác (demo bỏ nhãn vẫn giúp), hướng retrieval security-aware và H1 có cơ sở. Thiết kế này vì thế không đóng cửa các hướng cải tiến, mà đặt chúng sau một phép thử quyết định.

5. Ý tưởng tổng quan và lý lẽ đằng sau

Ý tưởng cốt lõi: **không thêm thành phần vào GRACE nữa, mà tháo từng thành phần ra và đo xem LLM có thực sự dùng nó không.** Mỗi thành phần (graph, nội dung demo, nhãn demo) được thay bằng một phiên bản *không mang thông tin* nhưng giữ nguyên hình thức và độ dài; nếu kết quả không đổi, LLM không dùng thành phần đó. Mọi so sánh được đo bằng điểm số liên tục lẫn nhãn, và

đặt cạnh hai mốc: mốc sàn (đối chứng hằng) và mốc có giám sát rẻ (hồi quy logistic trên embedding đóng băng).



khung chẩn đoán · 4 tầng, 5 probe, 2 mốc

Đầu vào đi qua bộ dựng prompt theo thang B0–B3; mỗi probe thay đúng một đầu vào bằng bản không mang thông tin; LLM và mốc có giám sát cùng đi vào một bộ đánh giá trên cùng danh sách ID.

5.1. Năm lý lẽ

Lý lẽ 1 – Quy kết tác dụng cần đối chứng thay thế, không chỉ đối chứng loại bỏ. Bài gốc trả lời RQ2 (vai trò graph) bằng cách bỏ graph khỏi prompt. Nhưng bỏ graph cũng làm prompt ngắn đi và đổi cấu trúc, nên không biết chênh lệch do *nội dung* graph hay do *sự có mặt* của một khối văn bản. Graph xáo trộn (lấy CPG của một hàm khác cùng kích thước) giữ nguyên hình thức và chỉ bỏ nội dung; đó mới là đối chứng đúng. Tương tự, demo ngẫu nhiên kiểm tra nội dung truy hồi; demo bỏ nhãn kiểm tra phần nhãn; đảo nhãn đo mức LLM bám vào nhãn.

Lý lẽ 2 – Điểm số liên tục tách "phân biệt" khỏi "ngưỡng". Khi LLM chỉ trả nhãn 0/1, một prompt làm mô hình đoán "1" nhiều hơn sẽ tăng recall và FP cùng lúc, và MCC có thể lên hoặc xuống tùy tỷ lệ lớp; đó chính là bế tắc của cặp tương phản trên dev set. Với vLLM, mỗi prompt cho logprob của token đầu tiên; điểm s = p(1)/(p(0)+p(1)) là một điểm số liên tục. AUROC/AUPRC trên s không phụ thuộc ngưỡng; nếu hai arm có AUROC bằng nhau nhưng MCC khác nhau, khác biệt chỉ là điểm hoạt động. Ngưỡng tối ưu MCC được chọn trên validation rồi áp sang test, nên mọi arm được so ở điểm hoạt động tốt nhất của chính nó.

Lý lẽ 3 – Hai mốc định vị ý nghĩa của mọi con số. Đối chứng hằng cho biết mức sàn do phân bố nhãn (F1 luôn-vulnerable = 2p/(1+p), bằng 63% trên Devign). Mốc có giám sát cho biết thông tin cần thiết *có nằm trong đầu vào hay không*: nếu một mô hình tuyến tính trên embedding CodeT5 đạt MCC 0,3 thì tín hiệu tồn tại trong mã nguồn, và việc LLM đạt 0,05 là lỗi của cách dùng LLM, không phải của dữ liệu. Mốc này rẻ (vài phút CPU) nên reviewer không thể bảo là so với một hệ thống đắt tiền.

Lý lẽ 4 – Tái lập ba backbone với chi phí một. Hai backbone API (Qwen3-Coder-480B, DeepSeek-V4.1-Flash) đã có kết quả full test; chỉ cần thêm backbone tự host để có logprob và các probe. Kết luận "không chuyển được" đứng trên ba model khác họ sẽ thuyết phục hơn nhiều so với một model.

Lý lẽ 5 – Chẩn đoán trước, giá trị gia tăng sau. Một hướng suy luận khác là bỏ qua các probe và đi thẳng vào câu hỏi "LLM bổ sung được gì cho bộ phân loại nhẹ" (learning to defer), lấy số ca LLM sửa đúng lỗi của bộ phân loại (303 trên Devign, 218 trên ReVeal) làm động cơ. Nhưng con số đó chỉ có nghĩa khi so với một expert không mang thông tin: với bộ phân loại đúng 1.784/2.732 mẫu Devign, một expert tung đồng xu (đúng 50%) kỳ vọng sửa được khoảng 474 lỗi và cho trần oracle ≈ 83%, cao hơn trần 76% tính từ full GRACE — nghĩa là 303 ca "sửa đúng" nằm dưới mức ngẫu nhiên, không phải bằng chứng có tín hiệu bổ sung. Vì vậy các phân tích giá trị gia tăng (P_pre/P_post, sửa–hỏng theo lớp, routing theo ngân sách) được giữ trong RQ5 nhưng luôn đi kèm đối chứng expert hằng và expert ngẫu nhiên cùng tỷ lệ đoán 1, và chỉ được diễn giải sau khi các probe ở RQ2–RQ4 cho biết LLM có đọc đầu vào hay không.

5.2. Vì sao đây là hướng ít tốn thời gian nhất

Thành phần cần cho bài	Trạng thái	Công việc còn lại
Pipeline tái triển khai, CPG cho 47.779 hàm, retrieval, prompt builder	Đã có (hai bản độc lập)	Thêm các probe: 4 hàm nhỏ trong prompt builder
Kết quả full test trên hai backbone API	Đã có	Tính lại bằng script chung, thêm CI
Mốc có giám sát và các arm phối hợp	Đã có (H3-B)	Chạy lại trên split đã khử trùng, 3 seed
Chấm mù retrieval, độ nhạy nhãn trên dev	Đã có	Mở rộng và chạy đảo nhãn trên full test
Script đánh giá, paired bootstrap, McNemar	Đã có	Thêm AUROC/AUPRC và chọn ngưỡng trên validation
LLM tự host có logprob	Chưa	Dựng vLLM, 1–2 ngày; chạy toàn bộ ma trận, 1–2 ngày GPU
Bài viết	Chưa	3 tuần
Cặp meta-model P_pre/P_post, sửa–hỏng theo lớp, routing theo ngân sách	Đã có (H3-B: 34 + 4 đặc trưng, benefit-router, OOF 3 fold)	Chạy lại offline với điểm s, một ngưỡng chung và đối chứng expert hằng/ngẫu nhiên: một ngày máy cá nhân

6. Phương pháp: giao thức thực nghiệm

Giao thức dưới đây là bản để đăng ký trước. Mọi thay đổi sau khi đăng ký phải được ghi lại kèm lý do.

6.1. Dữ liệu

Dataset	Train (kho retrieval)	Validation (chọn ngưỡng, chọn arm)	Test (kết luận)	Tỷ lệ vulnerable test	F1 luôn-vulnerable
Devign (DetectVul/devign)	21.854	2.724 (official validation)	2.732	45,9%	62,95%
ReVeal (SensorLLM/reveal)	18.187	2.196 (official validation sau lọc)	2.274	10,1%	18,37%
Big-Vul (chỉ bản tạp chí)	theo split 8:1:1 của bài gốc	—	—	5,9%	≈ 11%
PrimeVul (Ding et al., 2025; bản tạp chí, hoặc một mẫu khóa 300 + 300 nếu pipeline Joern còn dư thời gian)	official train	—	official test (cặp lỗi–vá)	≈ 3%	≈ 6%

Vệ sinh dữ liệu, làm một lần rồi đóng băng bằng manifest (ID + SHA-256 của mã đã chuẩn hóa):

1. Loại khỏi kho retrieval mọi hàm trùng chính xác với một hàm trong validation hoặc test (chuẩn hóa: bỏ khoảng trắng và chú thích). Kiểm toán sơ bộ đã đếm được 108 hàm Devign và 52 hàm ReVeal.
2. Gắn cờ gần trùng (Jaccard token ≥ 0,9) để phân tích độ nhạy, không loại.
3. Trong kho retrieval, giữ một đại diện cho nhóm mã trùng cùng nhãn; loại nhóm có nhãn mâu thuẫn (cách H3-B đã làm).
4. Ghi số mẫu từng lớp sau mỗi bước; bài báo báo cáo cả kết quả trên test đầy đủ lẫn test đã loại gần trùng.

6.2. Mô hình

Vai trò	Mô hình	Cách chạy	Dùng cho
LLM chính	Qwen3-Coder-30B-A3B-Instruct (MoE 30B, 3,3B tham số hoạt động; nhóm đã chạy thử bản GGUF Q4 trên Kaggle)	vLLM (bf16/FP8 trên GPU thuê) hoặc llama.cpp (GGUF Q4 trên Kaggle 2×T4); greedy, seed 42, logprob top-5 của token đầu	Mọi arm, mọi probe, điểm số liên tục
LLM dự phòng khi thiếu GPU	Qwen2.5-Coder-32B-Instruct (AWQ) hoặc 14B/7B	vLLM; cùng giao thức	Như trên; thêm trục kích thước cho bản tạp chí
LLM thứ	Qwen3-Coder-480B (Vertex), DeepSeek-V4.1-	Kết quả đã có, chỉ	RQ1, RQ3 (B0/B1/B2

Vai trò	Mô hình	Cách chạy	Dùng cho
hai, thứ ba	Flash (Vertex/FPT)	nhãn 0/1	và H2)
Mốc có giám sát	Hồi quy logistic trên CodeT5-base 768d (masked mean); MLP 768→128→1; hồi quy logistic chỉ trên 26 code metrics (SUP-METRIC, view numeric của H3-B)	CPU, 3 seed, ngưỡng chọn trên validation	RQ5
Retrieval	CodeT5 + whitening 256 → top-50 L2 → top-5 → rerank 0,7 Jaccard + 0,3 AST (theo mã tác giả)	Đã có	B2, B3
Graph	Joern CPG; ba cách đóng gói: prefix 25/25, lấp đầy ngân sách 1.500 token, H2 guarded cùng ngân sách	Đã có	RQ3

6.3. Các cấu hình (arm)

Mỗi arm khác arm gốc của nó đúng một điểm. Nhãn demo luôn là nhãn trong tập train; nhãn target không bao giờ vào prompt.

Mã	Nội dung prompt	Khác gì so với arm gốc	RQ
ALL-1, ALL-0, RAND	Không gọi LLM	Mốc sàn theo phân bố nhãn	Tất cả
SUP-LR, SUP-MLP, SUP-METRIC	Không gọi LLM	Mốc có giám sát trên embedding đóng băng; SUP-METRIC chỉ dùng 26 code metrics	RQ5
B0	vai trò + miền + code target + câu hỏi	Zero-shot, không graph, không demo	RQ1
B1	B0 + node/edge CPG, prefix 25/25	Thêm graph như bản tái triển khai	RQ1, RQ3
B1-full	B0 + CPG lấp đầy 1.500 token	Đổi chính sách cắt	RQ3
B1-H2	B0 + cụm evidence H2, cùng 1.500 token	Đổi cách đóng gói graph	RQ3
B1-shuf	B0 + CPG của một hàm khác cùng số nút (±10%), cùng ngân sách	Graph không mang thông tin	RQ3
B2	B1 + một demo truy hồi kèm nhãn (GRACE)	Thêm demo	RQ1, RQ2
B2-rand	B1 + một demo ngẫu nhiên từ kho, kèm nhãn thật của nó	Nội dung demo không liên quan	RQ2
B2-nolabel	B1 + demo truy hồi, không ghi nhãn	Bỏ nhãn demo	RQ2
B2-	B1 + một câu “Một hàm tương tự được	Chỉ còn gợi ý nhãn	RQ2

Mã	Nội dung prompt	Khác gì so với arm gốc	RQ
labelonly	gán nhãn X", không có mã demo		
B2-flip	B2 với nhãn demo đảo 1 ↔ 0	Đo độ nhạy nhãn	RQ2
B2-rep	B2 chạy lần hai, cùng prompt	Đo biến động tự nhiên	RQ2
B3	B1 + cặp demo tương phản (1 vulnerable, 1 safe) + khối mô tả khác biệt	Phương pháp contrastive ICL đã đề xuất	RQ2, RQ4
B3-nodiff	B3 không có khối mô tả khác biệt	Tách tác dụng khối mô tả	RQ2
B3-swap	B3 đảo thứ tự hai demo	Đo thiên lệch vị trí	RQ2
B3-flip	B3 đảo nhãn cả hai demo	Độ nhạy nhãn khi cân bằng	RQ2
P-PRE, P-POST	Không gọi LLM thêm: meta-LR trên 34 đặc trưng nền của H3-B, có/không 4 đặc trưng expert [s, valid, s·p, s−p]	LLM có bổ sung tín hiệu sau truy vấn không (cùng fold, cùng scaler, cùng lưới ngưỡng)	RQ5
ROUTE-rand / conf / benefit / oracle	Gọi LLM cho ceil(qN) mẫu chọn trước theo điểm pre-expert; q ∈ {5, 10, 25}%; random 20 seed	Có biết trước khi nào nên gọi LLM không	RQ5

6.4. Prompt và ngân sách context

- Thứ tự các phần theo Hình 6 của bài gốc và mã tác giả: vai trò + miền, code target, câu hỏi, thông tin nút, thông tin cạnh, demonstration ở cuối. Cách đặt demo lên trước trong một bản tái triển khai là một sai lệch so với cả hai nguồn và không dùng ở lượt cuối.
- Đầu ra: đúng một token, `0` hoặc `1`. Điểm số s = p(1)/(p(0)+p(1)) từ logprob top-5 của token đầu tiên. Nếu cả hai token không nằm trong top-5, đánh dấu invalid và báo cáo tỷ lệ (kỳ vọng dưới 0,5%); nếu smoke test cho thấy tỷ lệ này trên 1%, bật ràng buộc đầu ra (`guided_choice` của vLLM hoặc grammar của llama.cpp) và ghi rõ trong bài.
- Đơn vị duy nhất là token của tokenizer model. Ngân sách: code target ≤ 3.000 token, mỗi demo ≤ 1.500 token, graph ≤ 1.500 token, tổng ≤ 8.192 token. Ghi số token từng phần và cờ "bị cắt" cho mỗi prompt; không bao giờ cắt demo tới rỗng một cách im lặng — demo còn dưới 25% độ dài gốc bị đánh dấu và báo cáo riêng.
- Hàm thiếu CPG (Joern lỗi) không bị loại: mọi arm vẫn chạy trên hàm đó với cờ `graph_missing`, và tỷ lệ này được báo cáo như coverage.
- Kiểm tra toàn vẹn trước khi gọi LLM (prompt audit): phần giữ lại của code/demo/graph, số token từng phần, rò rỉ nhãn target, prompt trùng nhau; 30 prompt mỗi dataset chọn phân tầng theo độ dài và cờ cắt được hai người đọc bằng mắt.
- Temperature 0, greedy; seed 42. B2-rep đo phần không tất định còn lại do batching; mọi lượt gọi được cache theo hash của prompt, model và tham số, nên đổi model hay tham số không bao giờ dùng nhầm phản hồi cũ.

6.5. Chỉ số

Nhóm	Chỉ số	Ghi chú
Không ngưỡng	AUROC, AUPRC	AUPRC đọc cùng mốc ngẫu nhiên = tỷ lệ vulnerable
Có ngưỡng	MCC (chính), balanced accuracy, F1, precision, recall, tỷ lệ đoán 1, tỷ lệ invalid	Tính ở hai ngưỡng: $s \geq 0{,}5$ (câu trả lời của model) và τ^* tối ưu MCC trên validation
Mốc	ALL-1, ALL-0, RAND trong mọi bảng	F1 luôn-vulnerable = $2p/(1+p)$
Độ nhạy nhãn	Tỷ lệ đổi dự đoán giữa B2 và B2-flip; net = tỷ lệ đó trừ tỷ lệ đổi giữa B2 và B2-rep	Báo thêm đổi đúng→sai và sai→đúng
Giá trị bổ sung	ΔMCC và ΔAUROC($P_{post} - P_{pre}$); R_y, H_y theo lớp; trần oracle; MCC/BA/F1 theo ngân sách $q \in \{5, 10, 25\}$% cho random, confidence, benefit, oracle	Mọi số sửa/hỏng và trần oracle đặt cạnh expert hằng (ALL-0, ALL-1) và expert ngẫu nhiên cùng tỷ lệ đoán 1 (20 seed)
Chi phí	Ba nhóm: (a) chuẩn bị và học (embedding, huấn luyện mốc, meta-fit); (b) suy luận theo thuật toán (token/prompt, số lượt gọi theo ngân sách q); (c) chi phí vật lý của chiến dịch (GPU-giây/1.000 mẫu, cold so với warm cache, retry)	Cùng phần cứng; tỷ lệ gọi LLM 0% không có nghĩa chi phí 0

6.6. Thống kê

- Mọi so sánh là ghép cặp trên cùng danh sách ID. Khoảng tin cậy 95% cho ΔMCC, ΔAUROC, ΔF1 bằng paired bootstrap 5.000 lần với seed cố định; lưu chỉ số mẫu đã rút; lượt bootstrap làm metric không xác định (MCC mẫu số 0, AUROC thiếu một lớp) được đếm và báo cáo, không âm thầm bỏ.
- Kết luận đọc từ CI và biên δ theo bảng ở Mục 4.3; McNemar chỉ bổ sung cho bất đồng nhãn — nó so tỷ lệ lỗi, không so MCC, nên MCC có thể khác rõ mà McNemar không có ý nghĩa (và ngược lại); điều đó không mâu thuẫn.
- Mười một so sánh đăng ký trước ở Mục 4.3 là một họ kiểm định; dùng hiệu chỉnh Holm khi báo cáo p-value; p rất nhỏ ghi dạng bất đẳng thức ($p < 10^{-4}$), không ghi 0,0000.
- Thành phần ngẫu nhiên (demo ngẫu nhiên, graph xáo trộn, mốc có giám sát) chạy 3 seed (42, 43, 44); seed 42 là lượt chính khai báo trước, không chọn seed đẹp nhất; phản hồi LLM dùng lại qua ba seed của meta-model không được tính là ba lượt expert độc lập.
- Ba nguồn bất định được báo riêng: lấy mẫu (CI bootstrap), seed huấn luyện (ba seed), biến động phản hồi (B2-rep); không gộp thành một số ± chung.
- Nếu dataset có trường commit/project (Devign có commit_id), thêm cluster bootstrap theo commit và báo số cụm; nếu không, ghi rõ CI chưa xử lý phụ thuộc giữa các hàm cùng project.
- Expert invalid: P_{post} dùng đúng dự đoán của P_{pre} ở mẫu đó (fallback đối xứng) để hai arm chỉ khác nhau ở đặc trưng expert; mọi bảng ghi coverage và tỷ lệ invalid; không so một arm âm thầm bỏ ca lỗi với một arm dự đoán đủ mọi target.

6.7. Đăng ký trước và kiểm soát tiếp xúc test

Trước khi chạy test: commit một file `preregistration.md` gồm danh sách arm, ngân sách, chỉ số, giả thuyết và tiêu chí quyết định (bảng 4.3), kèm hash của manifest dữ liệu; đăng bản sao lên OSF. Ngưỡng τ* và mọi lựa chọn cấu hình chốt trên validation. Mỗi arm chạy test đúng một lần (trừ B2-rep có chủ đích). Bài báo ghi rõ rằng test set đã được xem trong giai đoạn thăm dò trước đăng ký; các kết quả lịch sử (36 run API, dev-100, H1–H3) chỉ xuất hiện như bằng chứng động cơ ở phụ lục, tách khỏi bảng xác nhận. Giao thức còn được lưu dạng máy đọc được (configs/protocol.yaml trong gói mã, Mục 8) để script đánh giá đọc đúng danh sách cặp so sánh, biên δ và seed đã khóa.

7. Kế hoạch thực nghiệm chi tiết (hands-on)

Mười bước, từ Bước 0 đến Bước 9, làm theo thứ tự. Mỗi bước nêu mục tiêu, lý do, ý nghĩa đối với bài báo, việc cần làm trên mã hiện có, file tương ứng trong gói mã `grace_diag_kit` (Mục 8) và đầu ra kiểm tra được. Thời lượng là ước tính cho một người làm toàn thời gian; Bước 0 và Bước 8 không cần GPU nên chạy song song với các bước khác.

Bước 0 – Phân tích offline từ dự đoán đã có (3 ngày đầu, không cần GPU)

Hạng mục	Nội dung
Mục tiêu	Tạo bảng canonical một dòng cho mỗi target của mọi run lịch sử, và dựng ba phân tích giá trị bổ sung từ dữ liệu H3-B hiện có: sửa–hỏng theo lớp với đối chứng, trần oracle, cặp P_pre/P_post đúng đối chứng.
Lý do	Khoảng 80% bằng chứng đã nằm trong artifact. Làm trước để biết sớm có tín hiệu bổ sung nào đáng theo đuổi không, và để hai pipeline dùng chung một evaluator trước khi tốn GPU.
Ý nghĩa	Cho RQ5 một kết quả sơ bộ và một bài học phương pháp (vì sao 303 ca “sửa đúng” chưa phải bằng chứng); là mốc nội bộ đầu tiên của kế hoạch.
Việc làm trên mã hiện có	Xuất `xpre_<split>.npz` và `gate_<split>.csv` từ `grace_h3v2/meta.py` (Mục 8, B1); ghép với dự đoán expert lịch sử thành CSV `id,y,p_gate,s_llm,valid` (expert 0/1 thì đặt s bằng nhãn, ghi rõ). Thống nhất một cách tính lại metric từ file từng mẫu cho cả hai pipeline (`common_evaluator.py` của repo và `grace_diag.stats`).
File trong gói mã	`grace_diag/complementarity.py`, `grace_diag/stats.py`
Đầu ra kiểm tra được	`runs/complementarity_<dataset>.json` gồm R_y/H_y theo lớp, hàng đối chứng ALL-0/ALL-1/RANDOM, trần oracle, ΔMCC(P_post − P_pre) với CI và kết luận theo δ; mọi số trong Mục 2.1 được tạo lại từ file từng mẫu với sai khác ≤ 0,001.

Bước 1 – Dựng LLM tự host có logprob (2–3 ngày)

Hạng mục	Nội dung

Hạng mục	Nội dung
Mục tiêu	Một server OpenAI-compatible trả logprob top-5 của token đầu tiên, tất định với seed 42.
Lý do	API thương mại không tất định, không có logprob và một endpoint đã ngừng giữa chừng; điểm số liên tục là điều kiện của RQ4 và của mọi so sánh không ngưỡng.
Ý nghĩa	Mọi arm chạy cùng một điều kiện phục vụ; ba nguồn bất định (lấy mẫu, seed, phản hồi) tách bạch được.
Việc làm	Chọn phương án phần cứng theo bảng dưới; khởi động server; chạy `run_matrix.py --dry-run` cho 20 prompt B2 của validation rồi `smoke_test_backend.py` (mỗi prompt gọi hai lần). Nếu dưới 99% prompt có token 0/1 trong top-5: bật `guided_choice` (vLLM) hoặc grammar (llama.cpp) trong `configs/backend_*.yaml` và chạy lại.
File trong gói mã	`configs/backend_vllm.yaml`, `configs/backend_llamacpp.yaml`, `scripts/smoke_test_backend.py`, `grace_diag/llm_backend.py`
Đầu ra kiểm tra được	`runs/smoke_test.json`: valid_rate ≥ 0,99, repeat_agreement ≥ 0,98, không lỗi; thông lượng (prompt/giây) để ước lượng Bước 5.

Phương án phần cứng	Model	Cách phục vụ	Chi phí ước tính
1 GPU 80 GB (A100/H100)	Qwen3-Coder-30B-A3B-Instruct bf16	vLLM	1,5–3 USD/giờ
1 GPU 48 GB (L40S, A6000 Ada)	Bản FP8 chính thức	vLLM	0,7–1,2 USD/giờ
1 GPU 24 GB (L4, RTX 4090)	AWQ 4-bit (kiểm tra vLLM hỗ trợ MoE) hoặc GGUF Q4	vLLM hoặc llama.cpp	0–0,5 USD/giờ
Kaggle 2×T4 (miễn phí)	GGUF UD-Q4_K_XL (17,7 GB) mà nhóm đã chạy thử	llama.cpp, chia layer hai GPU	0 USD; chậm hơn, ước 0,5–1 prompt/giây

```
pip install -e grace_diag_kit transformers
# GPU thuê
vllm serve Qwen/Qwen3-Coder-30B-A3B-Instruct --max-model-len 8192 --seed 42 --
enable-prefix-caching --port 8000
# hoặc Kaggle 2xT4
llama-server -m Qwen3-Coder-30B-A3B-Instruct-UD-Q4_K_XL.gguf --alias qwen3-coder-
30b-q4 \
  --n-gpu-layers all --split-mode layer --ctx-size 8192 --parallel 1 --port 8080 -
-seed 42
```

```
python scripts/smoke_test_backend.py --backend configs/backend_vllm.yaml \
  --prompts runs/dryrun/prompts_devign_validation.jsonl --n 20
```

Bước 2 – Đóng băng dữ liệu, kho retrieval và specs (2 ngày)

Hạng mục	Nội dung
Mục tiêu	Manifest có hash cho train/validation/test; kho donor đã khử trùng; file specs (code, CPG, demo B2, cặp B3, khối Key difference, evidence H2) cho validation và test.
Lý do	Hai cách đếm trùng lặp hiện cho hai kết quả khác nhau; mọi arm phải dùng cùng kho và cùng demo đã đóng băng; nhãn target phải nằm ngoài bộ dựng prompt.
Ý nghĩa	Điều kiện để mọi so sánh ghép cặp hợp lệ và để gói tái lập chạy lại được.
Việc làm	(1) Xuất JSONL thô `{id, func, target, nodes, edges, commit_id}` cho ba split từ kho CPG nén của repo; (2) `build_manifests.py` sinh manifest, kho donor và thống kê trùng; (3) chạy `prepare_verified_plan.py` của repo cho validation và test rồi `convert_verified_plan.py` để có `specs.jsonl`; (4) xuất `h2_evidence` từ pipeline H2 theo ngân sách 1.500 token cho arm B1-H2; (5) chọn validation subset 600 mẫu phân tầng, seed 42 (toàn bộ validation nếu GPU dư).
File trong gói mã	`scripts/build_manifests.py`, `scripts/convert_verified_plan.py`
Đầu ra kiểm tra được	`data/manifests/<dataset>_manifest_stats.json` (số mẫu từng lớp, trùng chính xác, gần trùng, nhóm nhãn mâu thuẫn, hash); `data/specs/<dataset>_<split>.jsonl`; hash manifest ghi vào `preregistration.md`.

Bước 3 – Bộ dựng prompt, các probe và kiểm tra toàn vẹn (3 ngày)

Hạng mục	Nội dung
Mục tiêu	Mười lăm arm từ một hàm duy nhất `render_arm(spec, arm, seed)`; mọi prompt được kiểm tra toàn vẹn trước khi gọi LLM.
Lý do	Quy kết tác dụng chỉ đúng khi mỗi arm khác arm gốc đúng một khối; hồ sơ cũ có demo bị cắt rỗng và prompt giống nhau nhưng kết quả khác nhau.
Ý nghĩa	Đây là "khung chẩn đoán" của bài; unit test và báo cáo audit là bằng chứng đưa cho reviewer.
Việc làm	Áp dụng quy tắc từng probe (danh sách dưới); chạy `pytest tests/test_probes.py`; `run_matrix.py --dry-run` cho mọi arm trên validation; `prompt_audit.py` và sửa cho đến khi rò rỉ nhãn bằng 0, demo rỗng ≤ 1%, graph thiếu được gắn cờ; hai

Hạng mục	Nội dung
	người đọc 30 prompt mỗi dataset trong `manual_sample.jsonl` (chọn phân tầng theo độ dài và cờ cắt).
File trong gói mã	`grace_diag/probes.py`, `grace_diag/token_budget.py`, `grace_diag/prompt_audit.py`, `tests/test_probes.py`, `configs/arms.yaml`
Đầu ra kiểm tra được	Test xanh; `audit_summary.json` theo arm: token p50/p95, tỷ lệ cắt code/graph/demo, demo rỗng, graph thiếu, rò rỉ nhãn = 0.

Quy tắc cho từng probe:

- **B1-shuf:** chọn tất định theo (id, seed) một hàm trong kho donor có số nút CPG trong khoảng ±10% số nút của target (nới dần nếu không có); dùng CPG của hàm đó với cùng chính sách cắt; code target giữ nguyên.
- **B1-full:** thêm nút theo thứ tự cho đến khi hết 1.500 token; cạnh chỉ giữ khi cả hai đầu đã được chọn. **B1-H2:** khối evidence của H2 thay cho khối nút/cạnh, cùng ngân sách.
- **B2-rand:** demo là một hàm donor ngẫu nhiên theo (id, seed), khác demo truy hồi, kèm nhãn thật của nó; không ép cân bằng.
- **B2-nolabel:** cùng demo với B2; câu dẫn đổi thành "The following is a related example." và không có từ chỉ nhãn.
- **B2-labelonly:** không có mã demo; chỉ một câu "A function similar to the one above was labelled Vulnerable/Non-vulnerable." với nhãn của demo B2.
- **B2-flip, B3-flip:** đảo mọi chỗ thể hiện nhãn (câu dẫn của từng demo và các từ chỉ nhãn trong khối Key difference của B3). **B3-swap:** đổi thứ tự hai demo. **B3-nodiff:** bỏ khối Key difference. **B2-rep:** cùng prompt B2, gọi lần hai không dùng cache.

Bước 4 – Đăng ký trước (1 ngày)

Hạng mục	Nội dung
Mục tiêu	`preregistration.md` và `configs/protocol.yaml` điền đủ, commit và tag `prereg-v1`, bản PDF đăng lên OSF.
Lý do	Test đã được xem; chỉ đăng ký trước mới cho các lượt test sau đó giá trị xác nhận.
Ý nghĩa	Tách bằng chứng động cơ khỏi bằng chứng xác nhận; trả lời trước câu hỏi của reviewer về tiếp xúc test.
Việc làm	Điền các ô `fill:` trong `protocol.yaml` (ngày, hash manifest, checkpoint model và SHA-256, phiên bản vLLM/llama.cpp); chép bảng 4.3 và các quy tắc Mục 6 vào `preregistration.md`; commit, tag. Từ đây mọi thay đổi ghi lý do trong `CHANGES_AFTER_PREREG.md`.
File trong gói mã	`configs/protocol.yaml`
Đầu ra kiểm tra được	Commit có tag; link OSF; mục 4 của checklist C được tích.

Bước 5 – Chạy ma trận cấu hình (3–5 ngày)

Hạng mục	Nội dung
Mục tiêu	Validation trước để chọn τ* cho từng arm; test chạy đúng một lần cho mọi arm theo ba tầng ưu tiên.
Lý do	Thứ tự tầng giúp kết quả dở dang vẫn dùng được; cache theo hash prompt + model + tham số giúp chạy lại không tốn GPU.
Ý nghĩa	Dữ liệu cho bảng A/B/C và cho RQ1–RQ4.
Việc làm	`run_matrix.py` với `--arms` theo tầng; seed 42 cho mọi arm, thêm 43 và 44 cho B2-rand và B1-shuf; theo dõi `records.jsonl` và thư mục `_cache/`; không chạy test của một arm trước khi τ* của arm đó đã được ghi vào `thresholds.json`.
File trong gói mã	`scripts/run_matrix.py`
Đầu ra kiểm tra được	`runs/final/<dataset>/<split>/<arm>/<seed>/records.jsonl` theo schema B.2 và `manifest.json` (model, backend, hash giao thức và specs, tokenizer).

Tầng	Arm	Trả lời	Số prompt ước tính (2 dataset)
1	B0, B1, B2, B2-flip, B2-rep	RQ1, lõi RQ2, RQ4	5 arm × (1.200 validation + 5.006 test) ≈ 31.000
2	B2-rand × 3 seed, B2-nolabel, B2-labelonly, B1-shuf × 3 seed	Cơ chế demo và graph	8 run × 6.206 ≈ 50.000
3	B1-full, B1-H2, B3, B3-nodiff, B3-swap, B3-flip	Đóng gói graph, cặp tương phản	6 run × 6.206 ≈ 37.000

Tổng khoảng 120.000 prompt, trung bình 2.500 token. Với vLLM trên A100 (khoảng 10.000 token/giây) là 8–10 giờ máy; với llama.cpp trên Kaggle 2×T4 cần chia Tầng 1 thành các phiên 9 giờ và dựa vào cache/resume. Các arm không gọi LLM (SUP-*, P-PRE/P-POST, ROUTE*-) tính ở Bước 6.

Bước 6 – Mốc có giám sát, cặp P_pre/P_post và routing (2 ngày)

Hạng mục	Nội dung
Mục tiêu	SUP-LR, SUP-MLP, SUP-METRIC với 3 seed và τ* trên validation; P_pre/P_post với điểm s của B2; bảng sửa–hỏng có đối chứng; đường chất lượng theo ngân sách.
Lý do	Mốc có giám sát định vị giá trị tuyệt đối của LLM; cặp P_pre/P_post là cách đúng duy nhất để hỏi "LLM có thêm tín hiệu không" khi đã có 34 đặc trưng nền; routing trả lời "có biết trước khi nào nên gọi LLM không".
Ý nghĩa	Trả lời RQ5 và các giả thuyết H5.1–H5.3; bảo vệ bài trước nhận

Hạng mục	Nội dung
	xét "so với hệ thống đắt tiền".
Việc làm	`supervised_anchor.py` với embedding CodeT5 768d của ba split và mã nguồn (cho SUP-METRIC); ghép `s`, `valid` của B2 với `p_gate` và `xpre` của H3-B rồi chạy `complementarity.py`; ghi CPU-giây huấn luyện và dự đoán 1.000 mẫu để so với GPU-giây của B2.
File trong gói mã	`grace_diag/supervised_anchor.py`, `grace_diag/complementarity.py`
Đầu ra kiểm tra được	`runs/anchor/anchor_scores.csv`, `anchor_summary.json`; `runs/complementarity_<dataset>.json` với kết luận theo δ cho H5.2 và H5.3.

Bước 7 – Thống kê, bảng và hình (3 ngày)

Hạng mục	Nội dung
Mục tiêu	Mọi bảng và hình của bài sinh từ một script, không sửa tay.
Lý do	Số liệu trong Mục 2 từng phải đối soát lại nhiều lần vì tính từ nhiều nguồn; một script duy nhất đọc `records.jsonl` loại bỏ nguyên nhân đó.
Ý nghĩa	Bảng B là nơi mỗi giả thuyết đăng ký trước nhận một trong bốn kết luận; đây là phần reviewer đọc kỹ nhất.
Việc làm	`evaluate_final.py` đọc `runs/final/`, `anchor_scores.csv` và `protocol.yaml`: Bảng A (mọi arm × dataset: AUROC, AUPRC, MCC/BA/F1 ở s ≥ 0,5 và ở τ*, tỷ lệ đoán 1, invalid, ba mốc hằng), Bảng B (các cặp đăng ký trước: Δ, CI 95%, số lượt bootstrap suy biến, McNemar, p Holm, kết luận theo δ), Bảng C (độ nhạy nhãn); Bảng D (giá trị bổ sung) lấy từ JSON của Bước 6; Hình 1 ROC của B0, B2, B3, SUP-LR; Hình 2 tỷ lệ đoán 1 so với MCC của các arm demo.
File trong gói mã	`grace_diag/evaluate_final.py`, `grace_diag/stats.py`
Đầu ra kiểm tra được	CSV và LaTeX của bốn bảng, hai hình, `thresholds.json`, và một trang đối chiếu với bảng giả thuyết 4.3.

Bước 8 – Chấm mù mở rộng, gói ca phân tầng và kiểm toán nhãn (1 tuần, song song với Bước 5–7)

Hạng mục	Nội dung
Mục tiêu	Dữ liệu định tính cho RQ6: độ liên quan cơ chế của demo truy hồi, cơ chế của các ca LLM và mốc có giám sát bất đồng, và tỷ lệ nhãn đáng ngờ.

Hạng mục	Nội dung
Lý do	Đây là dữ liệu ít bài nào có; nó giải thích các con số âm tính và định lượng trần do nhãn nhiễu.
Ý nghĩa	Nuôi phần Discussion và Threats to validity; là nền cho bản tạp chí.
Việc làm	(1) Mở rộng chấm mù retrieval lên 100 target mỗi dataset (thêm 50), cùng rubric bốn tiêu chí, tráo thứ tự, hai người chấm độc lập, người thứ ba phân xử; báo κ trước hòa giải; phân tích có điều kiện (accuracy của B2 trong nhóm demo liên quan so với không liên quan, nhóm nhãn demo trùng nhãn target so với khác). (2) `make_case_packets.py` tạo 60 ca theo bốn ô giữa SUP-LR và B2 (FN được sửa, FN bị tạo, FP được sửa, FP bị tạo; 15 ca mỗi ô), ẩn tên arm, phiếu có cột "không đủ ngữ cảnh" và "nhãn benchmark đáng ngờ". (3) Kiểm toán nhãn 50 FN + 50 FP của SUP-LR mỗi dataset; không sửa nhãn benchmark vì LLM giải thích thuyết phục — bảng nhãn gốc và bảng nhãn đã thẩm định (có bằng chứng patch/CVE) tách riêng.
File trong gói mã	`scripts/make_case_packets.py`
Đầu ra kiểm tra được	κ trước hòa giải cho từng bộ chấm; bảng phân tích có điều kiện với CI; bảng tỷ lệ nhãn hợp lý / đáng ngờ / không đủ ngữ cảnh.

Bước 9 – Gói tái lập (2 ngày)

Hạng mục	Nội dung
Mục tiêu	Một người ngoài nhóm tạo lại được mọi bảng từ gói, phần offline không cần GPU.
Lý do	Track RENE và EMSE chấm gói tái lập như một phần của bài.
Việc làm	Zenodo hoặc Figshare: mã ở commit đã tag, `protocol.yaml`, manifest, specs, `records.jsonl` với logprob, `anchor_scores.csv`, notebook sinh lại bảng và hình, README với lệnh chạy lại toàn bộ trong một ngày GPU; checksum của model, scaler và vocabulary; loại khóa API và đường dẫn cá nhân khỏi log; kiểm tra quyền phân phối dataset.
Đầu ra kiểm tra được	Gói chạy lại được từ README trên một máy sạch; DOI Zenodo ghi trong bài.

Điều kiện dừng và quy tắc quyết định trong quá trình làm:

- Nếu lợi thế của SUP-LR biến mất sau khử trùng hoặc kiểm toán nhãn: sửa dữ liệu trước, không viết.
- Nếu audit cho thấy graph hoặc demo không thực sự tới model (demo rỗng, graph thiếu trên 5%): sửa bộ dựng prompt và chạy lại arm liên quan; không kết luận từ một đầu vào bị mất phần chính.

- Nếu smoke test không đạt: chưa đăng ký trước cho đến khi backend ổn định.
- Nếu P_post không vượt P_pre dù có nhiều ca sửa đúng: xem CI, repeat và đối chứng ngẫu nhiên; không mặc định tăng độ sâu meta-model sẽ giải quyết.
- Không mở thêm model, ngưỡng hay arm sau khi đã xem test; mọi bổ sung được ghi "thăm dò" và để lại cho bản tạp chí.

8. Gói mã nguồn và hướng dẫn tích hợp vào phần mềm hiện tại

Gói `grace_diag_kit` (đính kèm cùng đề cương: khoảng 2.100 dòng Python, 29 unit test) không thay thế hai bản mã đang dùng. Nó nối vào ở ba điểm: lấy `prompt_spec` đã đóng băng từ repo tái triển khai, thay lớp gọi API bằng backend tự host có logprob, và đưa điểm số s cùng đặc trưng nền của H3-B vào phân tích giá trị bổ sung. Toàn bộ gói đã chạy thử đầu-cuối trên dữ liệu tổng hợp với một server giả lập OpenAI-compatible (manifest → dry-run → audit → ma trận → mốc có giám sát → giá trị bổ sung → bảng A/B/C → gói ca); chưa chạy trên dữ liệu thật của nhóm, nên bước đầu tiên khi nhận gói là `pip install -e grace_diag_kit` rồi `pytest grace_diag_kit/tests`.

8.1. Danh mục file

File	Vai trò	Dùng ở bước
`configs/protocol.yaml`	Giao thức đăng ký trước dạng máy đọc được: dataset, ngân sách token, arm theo tầng, seed, chỉ số, biên δ, 11 cặp so sánh, quy tắc dừng	4, 7
`configs/arms.yaml`	Định nghĩa 15 arm (graph, demo, nhãn, thứ tự, khối Key difference)	3, 5
`configs/backend_vllm.yaml`, `configs/backend_llamacpp.yaml`	Tham số server và request: temperature 0, max_tokens 1, top_logprobs 5, seed 42, guided_choice	1, 5
`src/grace_diag/probes.py`	`render_arm` theo Hình 6; `DonorPool` cho B2-rand và B1-shuf; `diff_blocks` cho kiểm thử	3
`src/grace_diag/token_budget.py`	Đếm và cắt theo tokenizer của model; cờ truncated	3
`src/grace_diag/llm_backend.py`	`FirstTokenScorer`: gọi vLLM hoặc llama.cpp, tính s, p0, p1, valid; cache theo hash(prompt, model, tham số, parser)	1, 5
`src/grace_diag/prompt_audit.py`	`audit_summary.json`, `audit_rows.csv`, `manual_sample.jsonl`	3
`src/grace_diag/supervised_anchor.py`	SUP-LR, SUP-MLP, SUP-METRIC; τ* trên validation; 3 seed	6
`src/grace_diag/complementarity.py`	`repair_harm`, `reference_experts`, `oracle_ceiling`, `fit_pre_post`/`apply_pre_post`, `routing_curves`	0, 6
`src/grace_diag/stats.py`	MCC (None khi suy biến), BA, F1,	0, 6, 7

File	Vai trò	Dùng ở bước
	AUROC/AUPRC, `paired_bootstrap` (từng mẫu hoặc theo cụm), McNemar, Holm, `pick_threshold`, `label_sensitivity`, `verdict` theo δ	
`src/grace_diag/evaluate_final.py`	Bảng A/B/C, hình ROC và điểm hoạt động, `thresholds.json`	7
`scripts/build_manifests.py`	Chuẩn hóa, SHA-256, trùng chính xác và gần trùng (MinHash), manifest, kho donor	2
`scripts/convert_verified_plan.py`	Plan schema v2 của repo → `specs.jsonl`	2
`scripts/run_matrix.py`	Chạy ma trận arm × seed × split; `--dry-run`; journal, resume, manifest run	3, 5
`scripts/smoke_test_backend.py`	20 prompt × 2 lần: valid_rate, repeat_agreement, thông lượng	1
`scripts/make_case_packets.py`	60 ca theo bốn ô, mã A/B tráo, answer key giữ riêng	8
`tests/`	29 test: mỗi probe đổi đúng một khối; không rò rỉ nhãn; ngân sách; thống kê; parser logprob	3, 7

8.2. Nối vào repo tái triển khai (thư mục `src/`, `scripts/`)

1. **Specs từ plan đã có, không sửa mã cũ.** `scripts/prepare_verified_plan.py` đang sinh plan schema v2 với `items[].prompt_spec` (query_code, nodes, edges, demonstrations, security_difference). Chạy nó cho official validation và test, rồi `convert_verified_plan.py --plan <plan> --out data/specs/<dataset>_<split>.jsonl --h2 <evidence H2>`; script gộp demo của arm b2 (một demo) và b3 (hai demo + Key difference) theo id và chỉ ghi nhãn vào trường `label`.
2. **Kho donor.** Từ kho CPG nén của train xuất JSONL `{id, func, target, nodes, edges}` rồi chạy `build_manifests.py`; file `<dataset>_donors.jsonl` chỉ giữ hàm train không trùng validation/test và không thuộc nhóm nhãn mâu thuẫn — đúng kho đã khử trùng mà B2 truy hồi.
3. **Backend tự host trong `src/evaluator.py`** (chỉ cần nếu muốn giữ một runner duy nhất; đường ngắn hơn là dùng `run_matrix.py` cho lượt mới và giữ `run_verified_experiment.py` cho các run API lịch sử). Trong `make_request_payload`, với provider `vLLM` hoặc `llama.cpp` thêm `logprobs=True, top_logprobs=5, seed=…` và `extra_body` cho guided_choice/grammar; trong `LLMEvaluator.__init__` thêm nhánh `_init_local_client()` với `openai.OpenAI(base_url=LOCAL_LLM_BASE_URL, api_key="EMPTY", max_retries=0)`; trong `generate_response` lưu `metadata["top_logprobs"]` từ `choice.logprobs.content[0].top_logprobs` (bọc try vì Vertex không trả trường này).
4. **Parser logprob trong `scripts/run_verified_experiment.py`.** `validate_plan` chấp nhận provider mới và `parser_policy = "logprob_binary_v1"`; trong `execute`, khi policy này được khai báo gọi

`grace_diag.llm_backend.parse_top_logprobs(metadata["top_logprobs"])` để lấy `pred_argmax`, `valid`, `s`, `p0`, `p1`, và ghi thêm vào record; các trường cũ (`llm_raw_response`, `finish_reason`, `usage`) giữ nguyên để `common_evaluator.py` vẫn reparse được.

5. **Thứ tự khối prompt.** `prompt_contract.render_control` đặt demonstration trước target và cắt theo ký tự; giữ nguyên nó để tái lập các run lịch sử. Lượt xác nhận dùng `grace_diag.probes.render_arm` (demo sau graph, cắt theo token). Nếu muốn `validate_payload_plan` kiểm lại prompt của lượt mới, thêm nhánh `payload_contract.renderer == "figure6_v2"` gọi `render_arm` thay cho `render_control`.
6. **Kiểm thử.** Chép `tests/test_probes.py` vào `tests/` của repo (đã có `conftest.py`) để mọi lần sửa bộ dựng prompt đều chạy lại kiểm tra "mỗi arm khác arm gốc đúng một khối".

8.3. Nối vào gói H3-B (`grace_h3v2`)

1. **Xuất đặc trưng nền và p_gate.** Trong `grace_h3v2/meta.py`, sau khi `fit_meta` tính `scores` (5 view) và `probability` (winner), lưu `pre_features(rows, scores, probability)` ra `xpre_<split>.npz` (mảng X 34 cột và `ids`) và `gate_<split>.csv` (`id, y, p_gate`) cho validation và test; test chỉ cần `p_gate`.
2. **Thay expert 0/1 bằng điểm s.** `expert_records` hiện lấy `prediction` 0/1 từ API; với lượt mới đọc `records.jsonl` của `run_matrix.py` (arm B2, seed 42) lấy `s` và `valid`. Ghép thành CSV `id,y,p_gate,s_llm,valid,group` cho mỗi split rồi chạy `python -m grace_diag.complementarity --val … --test … --xpre-val … --xpre-test … --tau <τ của gate chung> --out …`.
3. **Một ngưỡng chung.** Mọi policy trong bảng so sánh dùng cùng nhãn h từ cùng gate và cùng τ (ngưỡng chọn trên train OOF như `base['winner']` đang làm); gate_tuned (ngưỡng chọn trên validation) chỉ báo thêm dưới tên riêng; không chọn lại winner bằng test.
4. **Chiều của bảng sửa–hỏng.** Chiều chuẩn là bộ phân loại → expert: R = bộ phân loại sai và expert đúng, H = bộ phân loại đúng và expert sai. Bảng đối soát cũ tính theo chiều ngược (full GRACE → gate); khi dùng lại phải đổi chiều và luôn kèm hàng đối chứng expert hằng/ngẫu nhiên do `reference_experts` sinh.

8.4. Định dạng file trao đổi giữa hai pipeline và gói mã

File	Cột / trường	Ai tạo
`data/specs/<dataset>_<split>.jsonl`	id, query_code, nodes, edges, demonstrations, demonstrations_pair, security_difference, h2_evidence, label, group	`convert_verified_plan.py`
`data/manifests/<dataset>_donors.jsonl`	function_uid, func, target, nodes, edges	`build_manifests.py`
`runs/final/<dataset>/<split>/<arm>/<seed>/records.jsonl`	schema B.2 cộng s, p0, p1, top5, valid, tokens_*, truncated, group	`run_matrix.py`
`runs/anchor/anchor_scores.csv`	id, dataset, split, arm, seed, score	`supervised_anchor.py`

File	Cột / trường	Ai tạo
`<dataset>_<split>.csv` cho complementarity	id, y, p_gate, s_llm, valid, group	ghép từ H3-B và records
`runs/tables/table_{A,B,C}.csv`, `.tex`, `fig_*.png`	—	`evaluate_final.py`

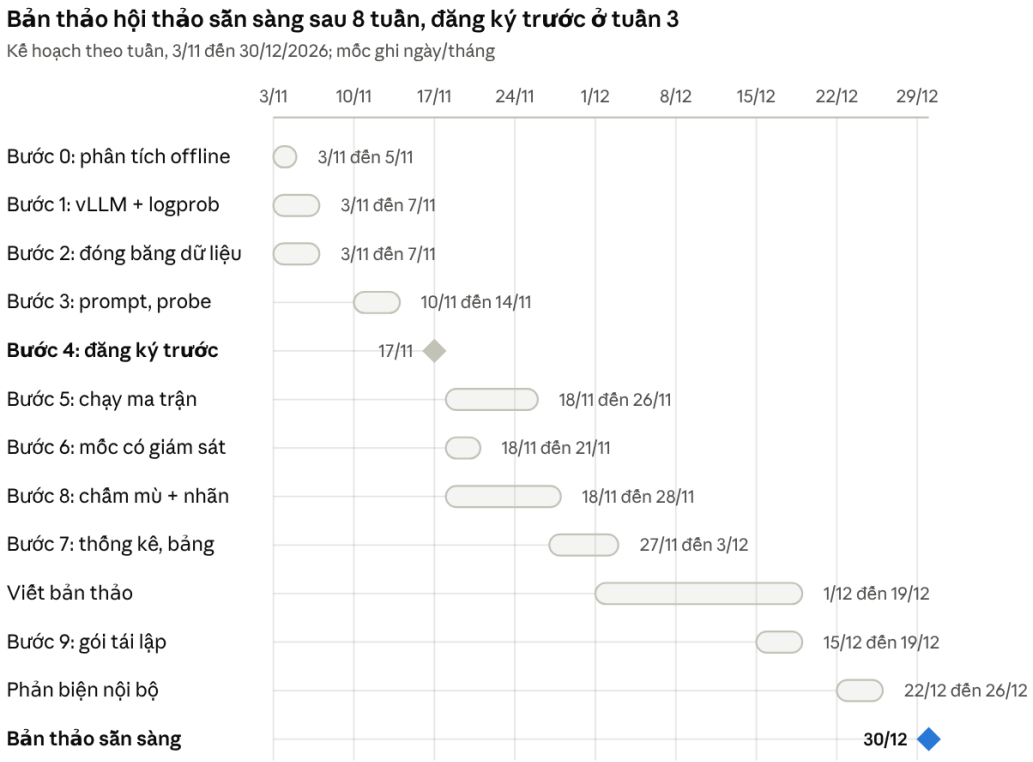
8.5. Trích cấu hình giao thức (`configs/protocol.yaml`)

```
backend:
  primary_model: Qwen/Qwen3-Coder-30B-A3B-Instruct
  decoding: {temperature: 0.0, max_tokens: 1, logprobs: true, top_logprobs: 5,
seed: 42}
token_budget: {total: 8192, target_code: 3000, demo: 1500, graph: 1500, reserve:
64}
prompt_order: [role_domain, target_code, question, node_info, edge_info,
demonstration, answer]
tiers:
  1: [B0, B1, B2, B2-flip, B2-rep]
  2: [B2-rand, B2-nolabel, B2-labelonly, B1-shuf]
  3: [B1-full, B1-H2, B3, B3-nodiff, B3-swap, B3-flip]
seeds: {headline: 42, all: [42, 43, 44]}
stats:
  n_boot: 5000
  bootstrap_seed: 20261117
  family_correction: holm
  margins: {mcc: 0.03, auroc: 0.02, ba: 0.01, mcc_meta: 0.02}
primary_contrasts:
  - {name: H1.1, a: B0, b: B2, metrics: [mcc, auroc]}
  - {name: H2.2a, a: B2, b: B2-rand, metrics: [auroc, mcc]}
  - {name: H3.1b, a: B1-shuf, b: B1, metrics: [auroc, mcc]}
  - {name: H5.3, a: P-PRE, b: P-POST, metrics: [mcc], margins: {mcc: 0.02}}
complementarity:
  expert_features: "[s, valid, s*p, s-p]; invalid -> s=0.5, valid=0; P_post
fallback ve P_pre"
  routing: {budgets: [0.05, 0.10, 0.25], random_seeds: 20, confidence: "|logit p -
logit tau|"}
```

Các ô `fill:` trong file thật (ngày đăng ký, hash manifest, SHA-256 của checkpoint, phiên bản server) phải được điền trước khi tag `prereg-v1`; `evaluate_final.py` đọc file này để biết danh sách cặp so sánh và biên δ, nên không thể "thêm một cặp so sánh" sau khi đã xem test mà không để lại dấu vết trong lịch sử commit.

9. Lộ trình, tổ chức công việc và chi phí

Bản thảo hội thảo có thể sẵn sàng ngày 30/12/2026 nếu bắt đầu ngày 3/11; mốc không được lùi là đăng ký trước ngày 17/11, vì mọi lượt chạy test sau đó mới có giá trị xác nhận.



lộ trình 8 tuần · 10 bước, 2 mốc

Bước 0, 1 và 2 chạy song song trong tuần đầu; chấm mù (Bước 8) chạy song song với ma trận (Bước 5) vì không cần GPU; viết bắt đầu khi có bảng Tầng 1, không đợi Tầng 3.

9.1. Tổ chức công việc

Công việc chia thành bốn mảng; hai mảng đầu tiến hành song song vì đều dựa trên mã đã có trong hai bản tái triển khai.

- **Hạ tầng và chạy thực nghiệm** (Bước 1, 3, 5, 9): vLLM, bộ dựng prompt, probe, ma trận cấu hình, gói tái lập. Điểm xuất phát: pipeline, bộ test, checkpoint, Joern và retrieval đã dựng.
- **Dữ liệu, mốc có giám sát và thống kê** (Bước 0, 2, 6, 7): manifest, SUP-LR/SUP-MLP/SUP-METRIC, cặp P_pre/P_post, routing theo ngân sách, script thống kê, bảng và hình. Điểm xuất phát: ledger 36 run, H3-B và paired bootstrap đã có; mã H2 chỉ cần xuất evidence theo ngân sách token chung để thành arm B1-H2.
- **Chấm mù và kiểm toán nhãn** (Bước 8): cần hai người chấm độc lập để tính κ và một người thứ ba phân xử các ca bất đồng. Điểm xuất phát: rubric và 100 target đã chấm.
- **Đăng ký trước, bản thảo, phản biện nội bộ** (Bước 4 và Mục 10): viết theo dàn ý Mục 10.1; phản biện nội bộ một tuần trước khi nộp.

Nhịp làm việc: một cuộc họp 45 phút mỗi thứ Sáu; trước họp, mỗi người cập nhật một file `PROGRESS.md` trong repo chung (việc xong, việc kẹt, số liệu mới). Không báo cáo dài; số liệu nằm trong bảng do script sinh.

9.2. Chi phí

Khoản	Ước tính	Ghi chú

Khoản	Ước tính	Ghi chú
GPU thuê cho Bước 1 và 5	20–40 giờ × 1–2 USD	Khoảng 30–60 USD; hoặc 0 USD với Kaggle 2×T4 và bản GGUF Q4 của Qwen3-Coder-30B-A3B mà nhóm đã chạy thử (chậm hơn, chia phiên 9 giờ, dựa vào cache/resume)
API trả phí	0	Kết quả Qwen3-480B và DeepSeek đã có; không chạy thêm
CPU cho mốc có giám sát, thống kê	Máy cá nhân	Vài giờ
Thời gian người	≈ 50 ngày công thực hiện và ≈ 6 ngày công phản biện nội bộ	Trong 8 tuần
Lưu trữ gói tái lập	Zenodo miễn phí	Dưới 5 GB

10. Kế hoạch viết bài và mục tiêu công bố

Bài hội thảo là một bài tái lập có chẩn đoán, 8–10 trang hai cột; bài tạp chí mở rộng theo bốn trục đã định sẵn. Tên làm việc (tiếng Anh): *Does GRACE Generalize Beyond GPT-4? A Diagnostic Replication of Graph-Enhanced In-Context Vulnerability Detection with Open-Weight LLMs*.

10.1. Dàn ý bài hội thảo

Mục	Nội dung	RQ	Bảng/hình
1. Introduction	GRACE và công thức graph + ICL; câu hỏi chưa được trả lời; ba đóng góp	—	—
2. Background and related work	GRACE; LLM cho phát hiện lỗ hổng; sao chép nhãn trong ICL; chất lượng nhãn dataset; tiêu chuẩn đánh giá (MCC); tái lập RAG4SVD; baseline code metrics; learning to defer	—	—
3. Replication protocol	Dữ liệu, mô hình, arm, probe, chỉ số, thống kê, đăng ký trước, sai lệch so với bài gốc và mã tác giả	—	Bảng arm; sơ đồ khung chẩn đoán
4. Results	4.1 Tái lập; 4.2 Cơ chế demo; 4.3 Graph; 4.4 Điểm số và ngưỡng; 4.5 Mốc có giám sát và giá trị bổ sung; 4.6 Truy hồi và lỗi	RQ1–RQ6	Bảng A, B, C; Hình ROC; Hình điểm hoạt động; bảng κ và phân tích có điều kiện
5. Discussion	Vì sao không chuyển được; hàm ý cho selective GRACE, các hướng retrieval và learning to defer; khuyến nghị đánh giá	—	—

Mục	Nội dung	RQ	Bảng/hình
6. Threats to validity	Tiếp xúc test, nhãn nhiễu, trùng lặp, khác model, không tất định của API	—	—
7. Conclusion	Kết luận và gói tái lập	—	—

Quy tắc viết kết quả: mỗi mục 4.x mở đầu bằng một câu trả lời RQ kèm số và CI; mọi bảng có ba mốc hằng và mốc có giám sát; không dùng từ "cải tiến" cho chênh lệch có CI chứa 0 và không dùng từ "không có tác dụng" khi cận trên CI chưa dưới δ; không đặt số dự kiến vào abstract; bảng thăm dò lịch sử (36 run API, dev-100) nằm ở phần động cơ hoặc phụ lục, tách khỏi bảng xác nhận; p rất nhỏ ghi dạng bất đẳng thức.

10.2. Nơi gửi

Loại	Nơi gửi	Vì sao phù hợp	Lưu ý
Hội thảo, ưu tiên 1	Track RENE (Reproducibility Studies and Negative Results) của ICSME 2027, SANER 2027 hoặc ICPC 2027	Track sinh ra để nhận tái lập và kết quả âm có phân tích; CORE A/B; reviewer chấm gói tái lập	Kiểm tra hạn nộp; thường giới hạn 10 trang
Hội thảo, ưu tiên 2	ESEM 2027 (Empirical Software Engineering and Measurement) hoặc MSR 2027 technical track	Bài thực nghiệm có đăng ký trước được đánh giá cao; ESEM có registered reports	Hạn nộp giữa năm
Tạp chí Q1, ưu tiên 1	Empirical Software Engineering (Springer)	Nhận replication và negative results có phân tích; yêu cầu gói tái lập	Bản mở rộng từ bài hội thảo ≥ 30% nội dung mới
Tạp chí Q1, ưu tiên 2	Journal of Systems and Software	Là nơi đăng GRACE; tái lập cùng tạp chí là thông lệ	Thư ngỏ với editor nêu rõ là tái lập
Tạp chí Q1, dự phòng	ACM TOSEM, Information and Software Technology	TOSEM có Replicated Computational Results	—
Hội thảo, ưu tiên 3	QRS 2027 (IEEE International Conference on Software Quality, Reliability and Security)	Nhận nghiên cứu thực nghiệm về an toàn phần mềm; các bài ensemble LLM cho phát hiện lỗ hổng gần đây đăng ở đây	Kiểm tra track và hạn nộp

Khi gọi một tạp chí là "Q1", kiểm tra SCIE và JCR quartile theo đúng năm và category (không đồng nhất với Scopus/SJR Q1); EMSE và JSS là đích tự nhiên, TSE/TOSEM đòi hỏi độ khái quát cao hơn. Không gọi một workshop đi kèm hội thảo lớn là bài main track; không có thiết kế nào bảo đảm được nhận — chất lượng bằng chứng và tính mới sau đối chiếu tài liệu quyết định.

10.3. Lộ trình nâng cấp thành bài tạp chí (3–4 tháng sau bài hội thảo)

1. **Dataset thứ ba và dữ liệu sạch hơn.** Big-Vul để hoàn tất tái lập cả ba dataset của bài gốc; PrimeVul (Ding et al., 2025) để có nhãn sạch hơn và các cặp hàm lỗi–đã vá. PrimeVul còn cho phép đo pairwise: mô hình có phân biệt được hàm lỗi với bản vá của chính nó không.
2. **Demo cặp vá lỗi thật.** Thay cặp tương phản heuristic hiện nay bằng cặp (hàm lỗi, bản vá) thật từ Big-Vul/PrimeVul, truy hồi theo tương đồng. Đây là demonstration tốt nhất có thể; nếu ngay cả nó cũng không giúp, giả thuyết ICL bị bác dứt điểm; nếu giúp, đó là đóng góp dương tính.
3. **Trục kích thước và họ model.** Qwen2.5-Coder 7B, 14B, 32B cùng giao thức; một model thương mại (GPT-4-class) trên subset 500 mẫu mỗi dataset để kiểm tra kết quả bài gốc có lặp lại ở model mạnh không.
4. **Kiểm toán nhãn và phân tích lỗi mở rộng.** 200 ca sai, hai người chấm, đối chiếu với commit vá thật; ước lượng trần hiệu năng do nhãn nhiễu.
5. **Mốc có giám sát mạnh hơn.** Tinh chỉnh CodeBERT hoặc UniXcoder (có số liệu công bố để đối chiếu) để đặt LLM vào đúng chỗ trên thang.
6. **Chi phí–chất lượng.** Đường cong MCC theo chi phí suy luận cho mọi phương pháp, trên cùng phần cứng.

11. Rủi ro và phương án dự phòng

Rủi ro lớn nhất không phải kỹ thuật mà là reviewer coi bài là "chỉ âm tính"; khung chẩn đoán, mốc có giám sát và đăng ký trước là câu trả lời cho rủi ro đó.

Rủi ro	Khả năng	Tác động	Phòng ngừa / dự phòng
Không thuê được GPU 48–80 GB	Trung bình	Chậm Bước 5	Dùng bản GGUF Q4 của Qwen3-Coder-30B-A3B trên Kaggle 2×T4 (đã chạy thử) hoặc Qwen2.5-Coder-14B AWQ; giảm validation subset còn 400; ưu tiên Tầng 1
Token `0`/`1` không nằm trong top-5 logprob ở nhiều prompt	Thấp	Điểm số không tin được	Bật guided_choice; kiểm tra ở smoke test trước khi đăng ký
Kết quả trên model tự host khác hẳn hai model API	Trung bình	Kết luận phức tạp hơn	Đây là phát hiện, không phải thất bại: báo cáo theo từng model và thảo luận
Một giả thuyết đăng ký bị bác (ví dụ LLM có đọc graph)	Trung bình	Đổi trọng tâm bài	Vẫn là kết quả công bố được; mở lại H2 như hướng cải tiến ở bản tạp chí
Reviewer chất vấn test đã được xem nhiều lần	Cao	Giảm độ tin	Khai báo rõ; đăng ký trước; thêm Big-Vul chưa từng chạm ở bản tạp chí
Nhãn nhiễu làm mọi phương pháp đều thấp	Cao	Khó diễn giải	Mốc có giám sát đạt MCC 0,3 cho thấy còn tín hiệu; kiểm toán nhãn ở Bước 8 định lượng trần
Trùng lặp train–test làm mốc có giám sát được	Trung bình	Phóng đại khoảng	Báo cả kết quả trên test đã loại trùng và gần trùng; nếu khoảng cách còn ≥ 0,15 MCC thì

Rủi ro	Khả năng	Tác động	Phòng ngừa / dự phòng
lợi		cách	kết luận giữ nguyên
Nhóm bận lịch học, tiến độ trượt	Cao	Lỡ hạn nộp	Tầng 1 đủ cho một bài RENE ngắn; Tầng 2–3 đưa sang bản tạp chí
Trùng ý tưởng với bài khác xuất hiện trong lúc làm	Thấp	Giảm tính mới	Tái lập độc lập luôn có giá trị ở track RENE; đăng ký trước có dấu thời gian
API DeepSeek/Qwen không tất định làm kết quả cũ khó bảo vệ	Trung bình	Backbone thứ hai, ba yếu hơn	Chỉ dùng chúng cho RQ1 và ghi rõ hạn chế; kết luận chính dựa trên model tự host
Bài bị xem là "đi sau" tái lập RAG4SVD (9/2026)	Trung bình	Giảm tính mới	Định vị là tái lập có chẩn đoán trên đúng benchmark gốc; trích dẫn và đối chiếu; đóng góp là cơ chế (nhãn demo, graph xáo trộn, logprob) và mốc có giám sát, không phải "tái lập đầu tiên"
Server tự host không trả logprob đúng định dạng (llama.cpp cũ, MoE + AWQ chưa được vLLM hỗ trợ)	Thấp	Chậm Bước 1	Nâng cấp llama.cpp/vLLM hoặc dùng bản bf16/FP8 trên GPU lớn hơn; smoke test bắt buộc trước đăng ký; dự phòng guided_choice/grammar

12. Các phương án suy luận khác đã xem xét và các điểm không nhất trí

Khi xây dựng đề cương, một phương án khác cho cùng bài toán đã được phân tích kỹ: lấy H3-B làm nền, đặt câu hỏi trung tâm là *giá trị gia tăng của nhánh LLM so với một bộ phân loại nhẹ đủ mạnh* (learning to defer), xác nhận bằng cặp meta-model có/không expert và phép so routing ở ngân sách 10% trên một cohort PrimeVul phân tầng khóa trước, với LLM gọi qua API như trước, trong 3–4 tuần. Mục này tổng kết những gì đã tiếp thu từ phương án đó và liệt kê đầy đủ bảy điểm không nhất trí kèm lý do, để người đọc tự đánh giá.

12.1. Những điểm đã tiếp thu hoặc dùng để hiệu chỉnh đề cương

Điểm	Đưa vào đâu
Cặp meta-model ghép đôi P_pre/P_post (cùng 34 đặc trưng nền, cùng fold, cùng lưới ngưỡng; fallback đối xứng khi expert invalid) thay cho STACK hai đặc trưng của bản trước	Mục 6.3, 6.6; Bước 6 (hiệu chỉnh)
Phân rã sửa–hỏng theo lớp (R_y, H_y) và trần oracle như trần chẩn đoán	Mục 6.5; Bước 0 và 6, có thêm đối chứng expert hằng/ngẫu nhiên
Đường chất lượng theo ngân sách gọi LLM: random 20 seed, confidence theo khoảng cách tới τ trong không gian logit, benefit-router, oracle chỉ gọi ca	RQ5, H5.3; Bước 6

Điểm	Đưa vào đâu
sửa đúng; batch routing top ceil(qN)	
Biên tác dụng δ khai báo trước và bảng bốn kết luận (lợi ích / loại trừ lợi ích / bất lợi / chưa đủ bằng chứng); tiêu chí "CI chứa 0" của bản trước không đủ để kết luận "không tác dụng"	Mục 4.3, 6.6 (hiệu chỉnh)
Một bộ phân loại, một ngưỡng, một bộ đặc trưng nền cho mọi phép so; không chọn lại winner bằng test	Mục 2.3 điểm 5; Mục 8.3
Kiểm tra toàn vẹn prompt trước khi gọi LLM: phần giữ lại, demo rỗng, graph thiếu, rò rỉ nhãn, 30 prompt đọc tay phân tầng	Mục 6.4; Bước 3
Expert của development và test phải cùng cấu hình phục vụ; cache theo identity đầy đủ; không dùng lại phản hồi của model khác	Mục 2.3 điểm 3; Mục 6.4
Tách ba nguồn bất định; lưu seed và chỉ số bootstrap; đếm lượt suy biến; McNemar không đo MCC; Holm cho họ kiểm định	Mục 6.6
Cluster bootstrap theo commit/project khi có; báo số cụm	Mục 6.6
Ba nhóm chi phí; tỷ lệ gọi LLM 0% không có nghĩa chi phí 0; cold so với warm cache	Mục 6.5
Pilot trên development trước full run; quy tắc dừng và cách xử lý kết quả âm	Bước 1; cuối Mục 7
Gói ca chấm mù phân tầng theo bốn ô sửa/hỏng; cho phép "không đủ ngữ cảnh" và "nhãn benchmark đáng ngờ"; không sửa nhãn vì LLM giải thích thuyết phục	Bước 8
Đối chiếu tài liệu cập nhật (RAG4SVD, Vul-RAG revisited, DLAP, CodeRAG, learning to defer, ensemble LLM, RALE, RevisitVD, code metrics, giao thức trên cặp lỗi–vá); sửa lại khoảng trống G1, G5 và đóng góp C1	Mục 1, 3, 4.2, 13 (hiệu chỉnh)
Kiểm tra SCIE/JCR quartile theo năm và category khi gọi "Q1"; không gọi workshop là main track; không đặt số dự kiến vào abstract; bảng thăm dò tách khỏi bảng xác nhận	Mục 10
Hai mảng việc dùng chung một pipeline đánh giá và kiểm chéo file join, nhãn; không ghép bảng từ hai	Mục 9.1; Bước 0

Điểm	Đưa vào đâu
pipeline độc lập ở cuối	

12.2. Bảy điểm không nhất trí

Mỗi điểm được trình bày như một khả năng suy luận hợp lý về bề ngoài, rồi phân tích vì sao không hợp lý trong bối cảnh và dữ liệu của nhóm; phần nào của ý tưởng vẫn được giữ cũng được nêu.

Điểm 1 – Lấy "giá trị gia tăng của LLM so với bộ phân loại nhẹ" (H3-B) làm trục chính của bài, thay cho tái lập có chẩn đoán. Lập luận của khả năng này: H3-B đã có embedding, classifier, dự đoán expert, router, meta-model và phần lớn công cụ đánh giá, nên "ít phải xây thêm nhất"; câu hỏi learning-to-defer sâu hơn việc thêm loss hay thêm prompt. Vì sao không theo: (i) câu hỏi đó giả định expert có tín hiệu để khai thác, trong khi mọi bằng chứng hiện có (MCC 0,02–0,10, độ nhạy nhãn 31%, trần oracle thấp hơn đồng xu) chỉ về phía LLM gần như không đọc đầu vào theo cách GRACE đưa vào; đo "khai thác" trước khi biết "có tín hiệu không" là đảo thứ tự suy luận; (ii) kết quả khả dĩ nhất (P_post ≈ P_pre, router ≈ random) là một kết quả âm không có cơ chế, dễ bị xem là "thêm một ensemble không thắng", và đã có tiền lệ gần (DLAP, learning to defer, bài code metrics ICSE 2026) nên tính mới hẹp; (iii) không thực sự ít việc hơn: cần cohort ngoài, pipeline Joern cho 2.000 hàm mới, expert chạy lại qua API và một backbone thứ hai; (iv) trục chẩn đoán bao hàm trục này: mọi phân tích giá trị gia tăng vẫn được làm (RQ5) nhưng với điểm số liên tục và đối chứng ngẫu nhiên, nên không mất gì. Phần được giữ: toàn bộ bộ công cụ của phương án này trở thành RQ5 và Bước 0, Bước 6.

Điểm 2 – Đọc số ca "expert sửa đúng lỗi của bộ phân loại" (303 trên Devign, 218 trên ReVeal) và trần oracle 76%/94% như bằng chứng rằng LLM có thông tin bổ sung đáng khai thác. Vì sao không hợp lý: với bộ phân loại đúng 65,3% trên Devign, một expert tung đồng xu kỳ vọng sửa 474 lỗi và cho trần oracle ≈ 83%; expert "luôn đoán 0" sửa toàn bộ FP của bộ phân loại. Số 303 và trần 76% vì thế thấp hơn mức của một expert không mang thông tin. Trên ReVeal (90% âm tính), expert ALL-0 có trần oracle rất cao chỉ do phân bố lớp. Kết luận: R, H và oracle chỉ có nghĩa khi đặt cạnh expert hằng và expert ngẫu nhiên cùng tỷ lệ đoán 1 — điều đề cương quy định bắt buộc. Phần được giữ: chính bảng R/H và trần oracle, nhưng luôn có đối chứng.

Điểm 3 – Tiếp tục dùng LLM qua API thương mại cho lượt xác nhận, chỉ cần "cùng provider và model cho development và test"; coi model open-weight tự host là lựa chọn đời cũ. Vì sao không theo: API không cho logprob (không có AUROC/AUPRC, không tách được khả năng phân biệt khỏi ngưỡng — chính là bế tắc của cặp tương phản), không tất định (6 target đổi nhãn với cùng prompt; 9.206 phản hồi rỗng), và endpoint có thể ngừng giữa chừng như đã xảy ra. Model tự host đề xuất (Qwen3-Coder-30B-A3B, 2025) không cũ; nhóm đã chạy thử trên Kaggle; hai backbone API vẫn giữ vai trò bằng chứng cho RQ1. Phần được giữ: yêu cầu cùng cấu hình phục vụ cho development và test — tự host thỏa mãn mặc định.

Điểm 4 – Dùng một cohort PrimeVul phân tầng (200 vulnerable + 1.800 safe), chuyển miền từ ReVeal, làm tầng xác nhận bắt buộc của bài hội thảo, với MCC có trọng số dân số làm outcome chính. Vì sao không theo cho bài hội thảo: (i) bài là tái lập GRACE nên phải đứng trên chính benchmark và giao thức của bài gốc; tiếp xúc test được xử lý bằng đăng ký trước, chọn mọi thứ trên validation và báo cả test đã khử trùng; (ii) cohort ngoài kéo theo pipeline Joern cho 2.000 hàm mới, kiểm toán chồng lấn với mọi nguồn cũ, trọng số lấy mẫu và CI phân tầng — khối lượng và rủi ro lớn hơn nhiều so với khung 3–4 tuần; (iii) chuyển miền ReVeal → PrimeVul trộn hai câu hỏi (GRACE có hoạt động không; có khái quát không) vào một con số, nếu lợi ích biến mất không biết do đâu. Phần

được giữ: PrimeVul là trục đầu tiên của bản tạp chí (Mục 10.3), và một mẫu khóa 300 + 300 được chạy cho Tầng 1 nếu Bước 2 còn dư thời gian.

Điểm 5 – Đặt phép so routing (benefit-router so với confidence ở ngân sách 10%) thành phép xác nhận chính thứ hai. Vì sao không theo: khi P_post không vượt P_pre và expert không hơn ngẫu nhiên, mọi router đều không có gì để khai thác; đặt nó làm xác nhận chính là đặt cược vào một tín hiệu chưa thấy. Routing được giữ như phân tích thứ cấp (H5.3) với random 20 seed và oracle làm hai cận; chi phí bằng không vì tính offline.

Điểm 6 – Chỉ đo tác động của "toàn gói graph + demo" (code-only so với GRACE đầy đủ) ở bài đầu; tách riêng graph và demonstration hay chạy ma trận đầy đủ là việc để sau. Vì sao không hợp lý: hai thành phần là hai tuyên bố riêng của bài gốc (RQ2 và RQ3 của GRACE) và có thể triệt tiêu nhau (graph làm giảm, nhãn demo làm tăng); phép so toàn gói không phân biệt được "LLM không đọc gì" với "đọc graph nhưng bị nhãn demo kéo". Với server tự host, các arm bổ sung chỉ tốn thêm vài giờ GPU, nên chi phí không phải lý do.

Điểm 7 – Lộ trình 3–4 tuần đến bài hội thảo. Vì sao không hợp lý: ước tính đó giả định artifact, endpoint và graph nguồn đều sẵn sàng, không tính thời gian hạ tầng lẫn viết bài; riêng cohort ngoài và backbone thứ hai mà chính phương án đó yêu cầu đã vượt khung này. Đề cương giữ 8 tuần nhưng tiếp thu ý "mức tối thiểu trong vài ngày": Bước 0 giao các phân tích offline trong ba ngày đầu.

12.3. Thống kê

Tổng cộng **7 quan điểm không nhất trí**: (1) trục chính là giá trị gia tăng trên H3-B; (2) diễn giải số ca sửa đúng và trần oracle như bằng chứng có tín hiệu; (3) giữ backbone API cho lượt xác nhận; (4) cohort PrimeVul phân tầng làm tầng xác nhận của bài hội thảo; (5) routing làm phép xác nhận chính; (6) chỉ đo toàn gói graph + demo; (7) lộ trình 3–4 tuần. Có **15 điểm được tiếp thu** (bảng 12.1), trong đó 3 điểm làm thay đổi nội dung của bản trước: biên δ và bảng bốn kết luận (Mục 4.3), cặp P_pre/P_post thay STACK (Mục 6.3), và việc sửa lại khoảng trống G1 cùng đóng góp C1 sau khi đối chiếu RAG4SVD.

13. Tài liệu tham khảo

Danh sách khởi điểm cho phần Related work; kiểm tra lại từng mục (năm, trang, DOI) trước khi nộp.

1. Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z. (2024). GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software*, 212, 112031. Mã nguồn: github.com/P-E-Vul/GRACE.
2. Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y. (2019). Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *NeurIPS 2019*.
3. Chakraborty, S., Krishna, R., Ding, Y., Ray, B. (2022). Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(9). (ReVeal; cũng chỉ ra trùng lặp trong Devign.)
4. Fan, J., Li, Y., Wang, S., Nguyen, T. N. (2020). A C/C++ code vulnerability dataset with code changes and CVE summaries. *MSR 2020*. (Big-Vul.)
5. Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y. (2025). Vulnerability detection with code language models: How far are we? *ICSE 2025*. (PrimeVul, đánh giá pairwise.)
6. Croft, R., Babar, M. A., Kholoosi, M. M. (2023). Data quality for software vulnerability datasets. *ICSE 2023*.

7. Risse, N., Böhme, M. (2024). Uncovering the limits of machine learning for automatic vulnerability detection. *USENIX Security 2024*.
8. Steenhoek, B., Rahman, M. M., Jiles, R., Le, W. (2023). An empirical study of deep learning models for vulnerability detection. *ICSE 2023*.
9. Steenhoek, B., Rahman, M. M., Roy, M. K., Alam, M. S., Barr, E. T., Le, W. (2024). A comprehensive study of the capabilities of large language models for vulnerability detection. arXiv:2403.17218.
10. Ullah, S., Han, M., Pujar, S., Pearce, H., Coskun, A., Stringhini, G. (2024). LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. *IEEE S&P 2024*.
11. Zhou, X., Zhang, T., Lo, D. (2024). Large language model for vulnerability detection: Emerging results and future directions. *ICSE 2024, NIER track*.
12. Gao, Z., et al. (2023). How far have we gone in vulnerability detection using large language models. arXiv:2311.12420. (VulBench.)
13. Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., Zettlemoyer, L. (2022). Rethinking the role of demonstrations: What makes in-context learning work? *EMNLP 2022*. (Sao chép nhãn trong ICL.)
14. Wei, J., Wei, J., Tay, Y., et al. (2023). Larger language models do in-context learning differently. arXiv:2303.03846. (Đảo nhãn demo.)
15. Rubin, O., Herzig, J., Berant, J. (2022). Learning to retrieve prompts for in-context learning. *NAACL 2022*. (EPR.)
16. Chicco, D., Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21, 6.
17. Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7). (McNemar.)
18. Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
19. Yamaguchi, F., Golde, N., Arp, D., Rieck, K. (2014). Modeling and discovering vulnerabilities with code property graphs. *IEEE S&P 2014*. (CPG, Joern.)
20. Wang, Y., Wang, W., Joty, S., Hoi, S. C. H. (2021). CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *EMNLP 2021*.
21. Fu, M., Tantithamthavorn, C. (2022). LineVul: A transformer-based line-level vulnerability prediction. *MSR 2022*. (Mốc tinh chỉnh cho bản tạp chí.)
22. Kwon, W., et al. (2023). Efficient memory management for large language model serving with PagedAttention. *SOSP 2023*. (vLLM.)
23. Nosek, B. A., Ebersole, C. R., DeHaven, A. C., Mellor, D. T. (2018). The preregistration revolution. *PNAS*, 115(11).
24. Tran, G. K., Vo, M. D. (2026). Deep learning-assisted pre-filtering for selective graph-structured LLM-based vulnerability detection. *APSIPA ASC 2026*. (Công trình kế thừa GRACE cần đối thoại trong Discussion; nếu cùng nhóm nghiên cứu, nêu rõ phần kế thừa.)
25. Kaniewski, S., Krämer, T., Bächle, J., Enzweiler, M., Menth, M., Heer, T. (2026). Retrieve, Reproduce, Reveal: Dissecting retrieval-augmented software vulnerability detection. arXiv:2609.37669. Gói tái lập: github.com/hs-esslingen-it-security/RAG4SVD. (Tái lập sáu hệ RAG, gồm GRACE, trên LLM open-weight; hiệu năng công bố không chuyển sang khung đánh giá chung.)
26. (2026). Revisiting Vul-RAG: Reproducibility and replicability of RAG-based vulnerability detection with open-weight models. arXiv:2606.04739. (Cùng nhóm với RAG4SVD; kiểm tra lại danh sách tác giả trước khi trích dẫn.)

27. Weissberg, F., Pirch, L., Imgrund, E., Möller, J., Eisenhofer, T., Rieck, K. (2026). LLM-based vulnerability discovery through the lens of code metrics. *ICSE 2026*. arXiv:2509.19117. (Bộ phân loại chỉ dùng code metrics ngang LLM tốt nhất; cơ sở cho SUP-METRIC.)
28. Yang, Y., et al. (2025). DLAP: A deep learning augmented large language model prompting framework for software vulnerability detection. *Journal of Systems and Software*, 219, 112234. arXiv:2405.01202.
29. Liu, Z., Chen, J., Huang, Z. (2026). Software vulnerability detection via multimodal retrieval and hierarchical decision making. *Science of Computer Programming*, 103491. (CodeRAG.)
30. Mozannar, H., Sontag, D. (2020). Consistent estimators for learning to defer to an expert. *ICML 2020*, PMLR 119.
31. Narasimhan, H., Jitkrittum, W., Menon, A. K., Rawat, A., Kumar, S. (2022). Post-hoc estimators for learning to defer to an expert. *NeurIPS 2022*.
32. Sun, Z., et al. (2025). Ensembling large language models for code vulnerability detection: An empirical evaluation. arXiv:2509.12629. (Preprint.)
33. Grasso, F. C., Liguori, P., Natella, R., Cotroneo, D. (2026). Enhancing LLM-based vulnerability detection via representation-aware logical ensemble (RALE). arXiv:2610.33756; QRS 2026.
34. Li, Y., Qi, W., Wang, X., Yu, F., Wang, X. (2026). Revisiting pre-trained language models for vulnerability detection. *AsiaCCS 2026*. arXiv:2507.16887. (Đánh giá, khái quát hóa, truncation.)
35. Cichoń, M., Dmitruk, B. (2026). A function-level vulnerability score measures flag rate more than the model: Protocol effects on paired benchmarks. arXiv:2609.32890. (Preprint; kiểm tra lại trước khi trích dẫn.)
36. Báo cáo nội bộ của nhóm (2026): báo cáo các hướng cải tiến GRACE H1, H2, H3; báo cáo tiến độ GRACE 22/10/2026; báo cáo chung 24/10/2026.

Phụ lục

A. Mẫu prompt (theo thứ tự Hình 6 của bài gốc)

Prompt đầy đủ của B2; các arm khác bỏ hoặc thay đúng một khối. Dòng cuối ép token đầu tiên của câu trả lời là chữ số.

```
You are now an excellent programmer. You are conducting a function vulnerability
detection task for C/C++ language.

[CODE SNIPPET]

In the above code snippet, check for potential security vulnerabilities and answer
with a single digit: 1 if Vulnerable, 0 if Non-vulnerable.

The node information of the code snippet is as follows:
[NODE INFORMATION]
The edge information of the code snippet is as follows:
[EDGE INFORMATION]

The following is an example, and it is [Vulnerable | Non-vulnerable].
[DEMONSTRATION CODE]

Answer (0 or 1):
```

Arm	Khối thay đổi
B0	Bỏ hai khối graph và khối example

Arm	Khối thay đổi
B1, B1-full, B1-H2, B1-shuf	Bỏ khối example; khối graph theo chính sách của arm
B2-nolabel	“The following is a related example.” + mã demo, không nhãn
B2-labelonly	“A function similar to the one above was labelled [Vulnerable | Non-vulnerable].”, không mã demo
B2-flip	Nhãn trong câu “and it is …” đảo ngược
B3	Hai khối: “The following example is Vulnerable.” + mã; “The following example is Non-vulnerable.” + mã; rồi khối “Key difference: …” do contrastive_selector sinh
B3-nodiff, B3-swap, B3-flip	Bỏ khối Key difference; đổi thứ tự hai khối; đảo cả hai nhãn (kể cả trong Key difference)

B. Mã tham khảo

B.1 – Gọi server tự host (vLLM hoặc llama.cpp) và tính điểm số s từ logprob của token đầu (rút gọn từ grace_diag/llm_backend.py):

```
import math
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY",
max_retries=0)  # llama.cpp: http://127.0.0.1:8080/v1
MODEL = "Qwen/Qwen3-Coder-30B-A3B-Instruct"
# llama.cpp: alias cua server
DIGIT = {"0": "0", "1": "1", " 0": "0", " 1": "1"}

def score(prompt, seed=42):
    r = client.chat.completions.create(model=MODEL, messages=[{"role": "user",
"content": prompt}],
                                       temperature=0, max_tokens=1, logprobs=True,
top_logprobs=5, seed=seed)
    top = {t.token: t.logprob for t in
r.choices[0].logprobs.content[0].top_logprobs}
    lp = {"0": None, "1": None}
    for tok, val in top.items():                          # gop bien the co khoang
trang ve "0"/"1"
        k = DIGIT.get(tok) or DIGIT.get(tok.strip())
        if k and (lp[k] is None or val > lp[k]):
            lp[k] = val
    valid = lp["0"] is not None or lp["1"] is not None  # ca hai vang mat ->
invalid, khong ep ve 0
    p0 = math.exp(lp["0"]) if lp["0"] is not None else 0.0
    p1 = math.exp(lp["1"]) if lp["1"] is not None else 0.0
    return {"s": p1 / (p0 + p1) if valid else float("nan"),
            "pred_argmax": int(p1 >= p0) if valid else None,
            "valid": valid, "raw": r.choices[0].message.content, "top5": top}
```

B.2 – Schema một dòng JSONL kết quả, đúng như run_matrix.py ghi (thêm p0, p1, top5, group, tokens_approx, latency_s, cached): `id`, `dataset`, `split`, `arm`, `seed`, `prompt_hash`, `demo_ids`,

`tokens_code`, `tokens_graph`, `tokens_demo`, `tokens_total`, `truncated` (cờ từng phần), `raw`, `top5`, `s`, `pred_argmax`, `valid`, `label`, `model`, `backend_kind`, `timestamp`.

B.3 – Khoảng tin cậy ghép cặp, McNemar và kết luận theo biên δ (rút gọn từ grace_diag/stats.py; bản đầy đủ có bootstrap theo cụm và đếm lượt suy biến):

```
import numpy as np
from scipy.stats import binomtest, chi2, rankdata

def mcc(y, p):
    tp = ((y == 1) & (p == 1)).sum(); tn = ((y == 0) & (p == 0)).sum()
    fp = ((y == 0) & (p == 1)).sum(); fn = ((y == 1) & (p == 0)).sum()
    den = np.sqrt(float((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn)))
    return None if den == 0 else (tp * tn - fp * fn) / den      # None = khong xac
dinh, khong ep 0

def auroc(y, s):                                                  # Mann-Whitney,
hoa chia doi
    n1 = int(y.sum()); n0 = len(y) - n1
    if n1 == 0 or n0 == 0:
        return None
    return (rankdata(s)[y == 1].sum() - n1 * (n1 + 1) / 2) / (n1 * n0)

def paired_ci(y, a, b, metric, n_boot=5000, seed=20261117, groups=None):
    """CI 95% cua metric(b) - metric(a) tren cung ID; groups != None -> rut theo
cum (commit)."""
    rng = np.random.default_rng(seed); n = len(y); d = []; degenerate = 0
    for _ in range(n_boot):
        if groups is None:
            idx = rng.integers(0, n, n)
        else:
            u, inv = np.unique(groups, return_inverse=True)
            pick = rng.integers(0, len(u), len(u))
            idx = np.concatenate([np.flatnonzero(inv == g) for g in pick])
        ma, mb = metric(y[idx], a[idx]), metric(y[idx], b[idx])
        if ma is None or mb is None:
            degenerate += 1; continue
        d.append(mb - ma)
    lo, hi = np.percentile(d, [2.5, 97.5])
    return metric(y, b) - metric(y, a), lo, hi, degenerate       # bao ca so luot
suy bien

def verdict(delta, lo, hi, margin):
    if lo > 0 and delta >= margin: return "benefit"
    if hi < 0: return "harm"
    if hi < margin: return "no_meaningful_benefit"                # can tren CI <
delta
    return "inconclusive"

def mcnemar(y, pa, pb):
    b = int(((pa == y) & (pb != y)).sum()); c = int(((pa != y) & (pb == y)).sum())
    if b + c < 25:
        return b, c, binomtest(min(b, c), b + c, 0.5).pvalue
    return b, c, chi2.sf((abs(b - c) - 1) ** 2 / (b + c), 1)

def pick_threshold(y_val, s_val):                                 # tau* toi uu
MCC tren validation
    grid = np.linspace(0.01, 0.99, 99)
```

```
    return max(grid, key=lambda t: ((mcc(y_val, (s_val >= t).astype(int)) or -2),
-abs(t - 0.5)))
```

B.4 – Graph xáo trộn: với mỗi target, lọc các hàm train có số nút CPG trong [0,9n; 1,1n], chọn một bằng `rng = np.random.default_rng(hash(target_id) % 2**32 + seed)`, dùng CPG của hàm đó qua cùng hàm định dạng và cùng ngân sách token.

B.5 – Đối chứng cho bảng sửa–hỏng và trần oracle (rút gọn từ grace_diag/complementarity.py); h là nhãn của mốc có giám sát, g là nhãn của LLM, chiều chuẩn là h → g:

```
def repair_harm(y, h, g):
    out = {}
    for cls in (0, 1):
        m = y == cls
        out[cls] = {"repair": int((m & (h != y) & (g == y)).sum()),   # h sai, g
dung
                    "harm":   int((m & (h == y) & (g != y)).sum())}   # h dung, g
sai
    return out

def oracle_accuracy(y, h, g):          # chon ben dung bang nhan that: chi la tran
chan doan
    return float(((h == y) | (g == y)).mean())

def reference_experts(y, h, positive_rate, n=20, seed=42):
    """Expert hang va expert ngau nhien cung ty le doan 1: moc so sanh bat buoc
cho R, H, oracle."""
    rng = np.random.default_rng(seed)
    rows = {name: (repair_harm(y, h, g), oracle_accuracy(y, h, g))
            for name, g in (("ALL-0", np.zeros_like(y)), ("ALL-1",
np.ones_like(y)))}
    rand = [(repair_harm(y, h, g), oracle_accuracy(y, h, g))
            for g in ((rng.random(len(y)) < positive_rate).astype(int) for _ in
range(n))]
    rows["RANDOM"] = rand              # bao trung binh va khoang 2,5-97,5% cua
repair, harm, oracle
    return rows
```

C. Checklist nghiệm thu trước khi viết

☐ Bước 0: metric lịch sử tạo lại từ file từng mẫu (sai khác ≤ 0,001); bảng sửa–hỏng có hàng ALL-0/ALL-1/RANDOM

☐ Smoke test backend: ≥ 99% prompt có token 0/1 trong top-5; hai lần chạy trùng ≥ 98%; thông lượng đã đo

☐ Manifest dữ liệu có hash; một kiểm toán trùng lặp duy nhất (chính xác và gần trùng); kho donor đã khử trùng

☐ `pytest tests/` xanh; prompt audit: rò rỉ nhãn = 0, demo rỗng ≤ 1%, graph thiếu được gắn cờ; 30 prompt mỗi dataset đã được hai người đọc

☐ `protocol.yaml` điền đủ; `preregistration.md` tag `prereg-v1` và đăng OSF trước lượt test đầu tiên

☐ τ* của mọi arm chọn trên validation, ghi trong `thresholds.json` trước khi chạy test

☐ Mọi run test có `records.jsonl` đủ schema B.2 và `manifest.json` môi trường; cache theo identity đầy đủ

☐ SUP-LR/SUP-MLP/SUP-METRIC 3 seed; P_pre/P_post cùng fold và fallback đối xứng; routing với random 20 seed và oracle

☐ Bảng A, B, C, D và hai hình sinh từ một script, không sửa tay; Bảng B có cột kết luận theo δ và số lượt bootstrap suy biến

☐ Bảng đối chiếu giả thuyết 4.3: ủng hộ / bị bác / chưa đủ bằng chứng, kèm số

☐ κ chấm mù (retrieval và gói 60 ca), bảng phân tích có điều kiện; kiểm toán nhãn 100 ca mỗi dataset

☐ Gói tái lập chạy lại được từ README trên một máy sạch; phần offline không cần GPU; không có khóa API trong log